Machine-checked certificate: Theorem 1

G. Adesso, *Optimality of Gaussian entanglement of formation*
arXiv:2608.01909v3

Lean 4 (v4.35.0-rc4) + Mathlib (commit `49fd9de`) · github.com/gerardo-adesso/gaussian-eof @ `ed019c9`

9 October 2026

Summary

All statements below are proved in Lean 4 against Mathlib, with no `sorry`, `admit` or additional axioms; the kernel reports that each depends only on `propext`, `Classical.choice` and `Quot.sound` (Sec. 7). Statements are printed verbatim from the source; the full source is in the Appendix.

Certified: the singular-value defect inequality (Theorem S1) in the Schmidt-basis matrix model; the geometric isotonic compression (Theorem S2), including the pool-adjacent-violators construction and Schur concavity of the entropy; the thermal closure (S:thermalF, S:support, S:Fmoments, S:UpsilonF); the assembly $E(\psi) + \lambda_{q,t}[D_t(\psi) - d_t(q)] \geq \mathcal{E}(q)$ for finite and infinite Schmidt rank; the endpoint $q = t$ and Eq. (S:fixedE) for $0 \leq q \leq t$. The definitions of $\mathcal{E}$, $\lambda_{q,t}$, d_t (`Thm2Cert/Reduction.lean`) and the 2×2 lemma `quad_nonneg` (`Thm2Cert/MatrixLemma.lean`) are shared with the Theorem 2 certificate.

Not formalized (inputs)

- The meta log-Sobolev theorem of Beigi and Rahimi-Keshari, used only for diagonal states of finite rank (`BRKUpsilon`), with the diagonal-state moments $\mathrm{Tr}(\rho_z a^\dagger a) = \bar{n}$, $\mathrm{Tr}(\sqrt{\rho_z} a \sqrt{\rho_z} a^\dagger) = T_z$.
- Fock-space facts: $E(\psi) = H(c^2)$, $D_t = G_t + 1 - t^2$, the matrix model $X = U^\dagger a U$, $Y = V^\dagger a V$ in Schmidt bases indexed by $\mathbb{N}$ (available for every finite-rank state), the index identities from $[a, a^\dagger] = 1$, and, for general infinite-rank states, the convergence $G_t(C^{(L)}) \to G_t(C)$ of normalized Schmidt truncations (then `theorem1_limit` applies).

1 Shared definitions (Thm2Cert)

`lam` — `Thm2Cert/Reduction.lean`

```
/-- `λ_{q,t} = −2 q ln q / [(t − q)(1 − t q)]`. -/
def lam (q t : ℝ) : ℝ := -2 * q * log q / ((t - q) * (1 - t * q))
```

`dt` — `Thm2Cert/Reduction.lean`

```
/-- `d_t(q) = (1 − t q)² / (1 − q²)`. -/
def dt (t q : ℝ) : ℝ := (1 - t * q) ^ 2 / (1 - q ^ 2)
```

`gfun` — `Thm2Cert/Reduction.lean`

```
/-- `g(y) = (y+1) ln(y+1) − y ln y`. -/
def gfun (y : ℝ) : ℝ := (y + 1) * log (y + 1) - y * log y
```

`EE` — `Thm2Cert/Reduction.lean`

```
/-- `ℰ(q) = g(q²/(1−q²))`, the entanglement of the TMSV. -/
def EE (q : ℝ) : ℝ := gfun (q ^ 2 / (1 - q ^ 2))
```

`quad_nonneg` — `Thm2Cert/MatrixLemma.lean`

```
lemma quad_nonneg {a c d : ℝ} (ha : 0 ≤ a) (hdet : d ^ 2 ≤ a * c) (hc : 0 ≤ c) (u v : ℝ) :
    0 ≤ a * u ^ 2 - 2 * d * u * v + c * v ^ 2
```

2 Definitions

pos — Thm1Cert/Defect.lean

```
/-- Positive part. -/
def pos (x : ℝ) : ℝ := max x 0
```

dd — Thm1Cert/Defect.lean

```
/-- The positive drops `d_n = (t c_n − c_{n+1})₊`. -/
def dd (n : ℕ) : ℝ := pos (t * c n - c (n + 1))
```

dlt — Thm1Cert/Defect.lean

```
/-- Truncated positive drops: `δ_k = d_k` for `k ≤ N`, `0` otherwise. -/
def dlt (N k : ℕ) : ℝ := if k ≤ N then dd t c k else 0
```

alpha — Thm1Cert/Defect.lean

```
/-- `α_i = ∑_{k ≥ i} t c_k δ_k`. -/
def alpha (N i : ℕ) : ℝ := ∑ k ∈ Ico i (N + 1), t * c k * dlt t c N k
```

beta — Thm1Cert/Defect.lean

```
/-- `β_i = −∑_{k ≥ i} c_{k+1} δ_k`. -/
def beta (N i : ℕ) : ℝ := -∑ k ∈ Ico i (N + 1), c (k + 1) * dlt t c N k
```

wt — Thm1Cert/Compression.lean

```
/-- Weights `w_n = t^{2n}`. -/
def wt (n : ℕ) : ℝ := (t ^ n) ^ 2
```

mass — Thm1Cert/Compression.lean

```
/-- Total weighted mass `∑_{n ≤ M} w_n ℓ_n`. -/
def mass (ℓ : ℕ → ℝ) : ℝ := ∑ n ∈ range (M + 1), wt t n * ℓ n
```

pre — Thm1Cert/Compression.lean

```
/-- Prefix mass `∑_{n ≤ K} w_n ℓ_n`. -/
def pre (ℓ : ℕ → ℝ) (K : ℕ) : ℝ := ∑ n ∈ range (K + 1), wt t n * ℓ n
```

en — Thm1Cert/Compression.lean

```
/-- Drop energy (S:senergy): `∑_{n ≤ M} (n+1) t^{2n+2} ((√ℓ_n − √ℓ_{n+1})₊)²`. -/
def en (ℓ : ℕ → ℝ) : ℝ :=
  ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t ^ (n + 1)) ^ 2 * pos (√(ℓ n) - √(ℓ (n + 1))) ^ 2
```

Hs — Thm1Cert/Closure.lean

```
/-- Entropy `H(z²) = ∑ −z_n² ln z_n²` of a square-root sequence. -/
def Hs (z : ℕ → ℝ) : ℝ := ∑' n, negMulLog (z n ^ 2)
```

Ft — Thm1Cert/Closure.lean

```
/-- `𝓕_t(z) = ∑ (n+1)(t z_n − z_{n+1})²` (Eq. S:Fdef). -/
def Ft (t : ℝ) (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2
```

zs — Thm1Cert/Closure.lean

```
/-- Thermal square-root sequence `z^{(s)}_n = √(1−s²) sⁿ` (Eq. S:thermalz). -/
def zs (s : ℝ) (n : ℕ) : ℝ := √(1 - s ^ 2) * s ^ n
```

phi Thm1Cert/Closure.lean

```
/-- `φ(s) = 𝓔(s) + λ (t−s)²/(1−s²)`. -/
def phi (L t s : ℝ) : ℝ := EE s + L * ((t - s) ^ 2 / (1 - s ^ 2))
```

nbar Thm1Cert/Upsilon.lean

```
/-- Mean photon number `n̄ = ∑ n z_n²`. -/
def nbar (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, (n : ℝ) * z n ^ 2
```

Tz Thm1Cert/Upsilon.lean

```
/-- `T_z = ∑ (n+1) z_n z_{n+1} = Tr(√ρ_z a √ρ_z a†)` (Eq. S:Tz). -/
def Tz (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, ((n : ℝ) + 1) * z n * z (n + 1)
```

Ups Thm1Cert/Upsilon.lean

```
/-- `Y(ρ_z)` for `ν₀ = λt`, `ν₁ = 0`, `ω = λ(1−t)²` (Eqs. S:Upsilon, S:parameters). -/
def Ups (t L : ℝ) (z : ℕ → ℝ) : ℝ :=
  2 * (L * t) * ((nbar z + 1) - Tz z) + L * (1 - t) ^ 2 * nbar z + Hs z
```

BRKUpsilon Thm1Cert/Upsilon.lean

```
/-- **Input (Beigi–Rahimi-Keshari)**, restricted to diagonal states of finite rank:
`Y(ρ_z) ≥ inf_{0 ≤ s < 1} Y(τ_s)`. -/
def BRKUpsilon (t L : ℝ) : Prop :=
  ∀ z : ℕ → ℝ, (∀ n, 0 ≤ z n) → (∃ M, ∀ n, M < n → z n = 0) → HasSum (fun n => z n ^ 2) 1 →
    ∀ c0 : ℝ, (∀ s, 0 ≤ s → s < 1 → c0 ≤ Ups t L (zs s)) → c0 ≤ Ups t L z
```

BRK Thm1Cert/Assembly.lean

```
/-- **Input (Beigi–Rahimi-Keshari, diagonal states of finite rank).** For every finitely
supported `z ≥ 0` with `∑ z_n² = 1`, `H(z²) + λ 𝓕_t(z)` is at least the infimum over thermal
states. (Y(ρ_z) = H(z²) + λ𝓕_t(z) + const for the parameters (S:parameters).) -/
def BRK (t L : ℝ) : Prop :=
  ∀ z : ℕ → ℝ, (∀ n, 0 ≤ z n) → (∃ M, ∀ n, M < n → z n = 0) → HasSum (fun n => z n ^ 2) 1 →
    ∀ c0 : ℝ, (∀ s, 0 ≤ s → s < 1 → c0 ≤ Hs (zs s) + L * Ft t (zs s)) → c0 ≤ Hs z + L * Ft t z
```

3 Theorem S1: singular-value defect inequality

drop_lemma Thm1Cert/Defect.lean

```
/-- `D_ij = ∑_{k=i}^{j-1} (t c_k − c_{k+1})₊ ≤ (t c_i − c_j)₊` for `i ≤ j`. -/
theorem drop_lemma {t : ℝ} {c : ℕ → ℝ} (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (hc : Antitone c)
    (hc0 : ∀ n, 0 ≤ c n) {i j : ℕ} (hij : i ≤ j) :
    ∑ k ∈ Ico i j, dd t c k ≤ pos (t * c i - c j)
```

pointwise Thm1Cert/Defect.lean

```
/-- **Eq. (S:pointwise)**: `|t c_i x − c_j y|² ≥ (α_i − α_j)|x|² + (β_i − β_j)|y|²`. -/
theorem pointwise (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n)
    (N i j : ℕ) (x y : ℂ) :
    (alpha t c N i - alpha t c N j) * ‖x‖ ^ 2 + (beta t c N i - beta t c N j) * ‖y‖ ^ 2
      ≤ ‖((t * c i : ℝ) : ℂ) * x - (c j : ℂ) * y‖ ^ 2
```

defect_inequality Thm1Cert/Defect.lean

```
/-- **Theorem S1** (singular-value defect inequality), matrix model.
`hG`: `G = ∑_{i,j} |t c_i X_ij − c_j Y_ij|²`; `hX`, `hY`: the index identities (S:index),
required only for indices with `c_i > 0`. Conclusion: `∑_{n ≤ N} (n+1) d_n² ≤ G` for every `N`,
hence `∑_n (n+1) d_n²` converges and is `≤ G`. -/
theorem defect_inequality (ht0 : 0 < t) (ht1 : t ≤ 1) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n)
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) (N : ℕ) :
    ∑ k ∈ range (N + 1), ((k : ℝ) + 1) * dd t c k ^ 2 ≤ G
```

defect_inequality_tsum Thm1Cert/Defect.lean

```
/-- Consequence: the defect series converges and `∑_n (n+1) d_n² ≤ G_t(C)`. -/
theorem defect_inequality_tsum (ht0 : 0 < t) (ht1 : t ≤ 1) (hc : Antitone c)
    (hc0 : ∀ n, 0 ≤ c n) (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Summable (fun n : ℕ => ((n : ℝ) + 1) * dd t c n ^ 2) ∧
    ∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2 ≤ G
```

4 Theorem S2: geometric isotonic compression

pool_step Thm1Cert/Compression.lean

```
/-- **Pooling step.** If two adjacent levels violate monotonicity, pooling the two maximal
constant runs around them preserves the invariants and strictly reduces the number of breaks. -/
theorem pool_step (ht : 0 < t) {r ℓ : ℕ → ℝ} (hI : Inv M t r ℓ) {n : ℕ} (hn : n < M)
    (hv : ℓ n < ℓ (n + 1)) : ∃ ℓ', Inv M t r ℓ' ∧ brk M ℓ' < brk M ℓ
```

pava_exists Thm1Cert/Compression.lean

```
/-- **Pool-adjacent-violators, existence form.** There are levels satisfying the invariants
that are nonincreasing on `[0, M]`. -/
theorem pava_exists (ht : 0 < t) {r : ℕ → ℝ} (hI : Inv M t r r) :
    ∃ ℓ, Inv M t r ℓ ∧ ∀ n, n < M → ℓ (n + 1) ≤ ℓ n
```

entropy_le_of_prefix Thm1Cert/Compression.lean

```
/-- If `p` is positive and nonincreasing on `[0, M]` and every prefix sum of `q ≥ 0` dominates
that of `p`, with equal totals, then `H(q) ≤ H(p)`. -/
theorem entropy_le_of_prefix (p q : ℕ → ℝ) (hp : ∀ n, n ≤ M → 0 < p n)
    (hanti : ∀ n, n < M → p (n + 1) ≤ p n) (hq : ∀ n, 0 ≤ q n)
    (hpre : ∀ K, K ≤ M → ∑ n ∈ range (K + 1), p n ≤ ∑ n ∈ range (K + 1), q n)
    (htot : ∑ n ∈ range (M + 1), p n = ∑ n ∈ range (M + 1), q n) :
    ∑ n ∈ range (M + 1), negMulLog (q n) ≤ ∑ n ∈ range (M + 1), negMulLog (p n)
```

compression Thm1Cert/Compression.lean

```
/-- **Theorem S2** (geometric isotonic compression). -/
theorem compression (ht : 0 < t) (c : ℕ → ℝ) (hcpos : ∀ n, n ≤ M → 0 < c n)
    (hanti : ∀ n, n < M → c (n + 1) ≤ c n) (hsupp : ∀ n, M < n → c n = 0)
    (hnorm : ∑ n ∈ range (M + 1), c n ^ 2 = 1) :
    ∃ z : ℕ → ℝ, (∀ n, 0 ≤ z n) ∧ (∀ n, M < n → z n = 0) ∧
      ∑ n ∈ range (M + 1), z n ^ 2 = 1 ∧ (∀ n, z (n + 1) ≤ t * z n) ∧
      ∑ n ∈ range (M + 1), negMulLog (z n ^ 2) ≤ ∑ n ∈ range (M + 1), negMulLog (c n ^ 2) ∧
      ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2
        ≤ ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * pos (t * c n - c (n + 1)) ^ 2
```

5 Thermal closure

H_thermal — Thm1Cert/Closure.lean

```
/-- **Eq. (S:thermalF), entropy part**: `H((z^{(s)})²) = 𝓔(s)`. -/
theorem H_thermal {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) : Hs (zs s) = EE s
```

F_thermal — Thm1Cert/Closure.lean

```
/-- **Eq. (S:thermalF), energy part**: `𝓕_t(z^{(s)}) = (t−s)²/(1−s²)`. -/
theorem F_thermal (t : ℝ) {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) :
    Ft t (zs s) = (t - s) ^ 2 / (1 - s ^ 2)
```

support — Thm1Cert/Closure.lean

```
/-- **Eq. (S:support)**: for `0 < q < t ≤ 1` and `λ = λ_{q,t}`,
`𝓔(s) + λ(t−s)²/(1−s²) ≥ 𝓔(q) + λ(t−q)²/(1−q²)` for all `0 ≤ s < 1`. -/
theorem support {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) {s : ℝ} (hs0 : 0 ≤ s)
    (hs1 : s < 1) : phi (lam q t) t q ≤ phi (lam q t) t s
```

Fmoments — Thm1Cert/Upsilon.lean

```
/-- **Eq. (S:Fmoments)** in `HasSum` form. -/
lemma Fmoments (t : ℝ) (z : ℕ → ℝ) {N T : ℝ} (h1 : HasSum (fun n => z n ^ 2) 1)
    (hN : HasSum (fun n : ℕ => (n : ℝ) * z n ^ 2) N)
    (hT : HasSum (fun n : ℕ => ((n : ℝ) + 1) * z n * z (n + 1)) T) :
    HasSum (fun n : ℕ => ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2)
      (t ^ 2 * (N + 1) + N - 2 * t * T)
```

Ups_eq — Thm1Cert/Upsilon.lean

```
/-- **Eq. (S:UpsilonF)**: `Υ(ρ_z) = H(z²) + λ𝓕_t(z) + λ(2t − t²)`. -/
lemma Ups_eq (t L : ℝ) (z : ℕ → ℝ) (h1 : HasSum (fun n => z n ^ 2) 1)
    (hN : Summable (fun n : ℕ => (n : ℝ) * z n ^ 2))
    (hT : Summable (fun n : ℕ => ((n : ℝ) + 1) * z n * z (n + 1))) :
    Ups t L z = Hs z + L * Ft t z + L * (2 * t - t ^ 2)
```

brk_of_upsilon — Thm1Cert/Upsilon.lean

```
/-- The Beigi–Rahimi-Keshari input in Υ form implies the sequence form `BRK`. -/
theorem brk_of_upsilon {t L : ℝ} (hU : BRKUpsilon t L) : BRK t L
```

6 Theorem 1 and corollaries

thm1_spectrum — Thm1Cert/Assembly.lean

```
/-- **Theorem 1, spectral form.** -/
theorem thm1_spectrum (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (hD : Summable (fun n : ℕ => ((n : ℝ) + 1) * dd t c n ^ 2)) :
    phi (lam q t) t q
      ≤ ∑' n, negMulLog (c n ^ 2) + lam q t * ∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2
```

theorem1 Thm1Cert/Assembly.lean

```
/-- **Theorem 1** (affine extremality), matrix model. With `E(ψ) = H(c²)` (Eq. S:entropy) and
`D_t(ψ) = G + 1 − t²` (Eq. S:Gdef), where `G = ∑ |t c_i X_ij − c_j Y_ij|²` (Eq. S:Gmatrix):
`E(ψ) + λ_{q,t} [D_t(ψ) − d_t(q)] ≥ 𝓔(q)` for all `0 < q < t ≤ 1`. -/
theorem theorem1 (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    EE q ≤ ∑' n, negMulLog (c n ^ 2) + lam q t * ((G + 1 - t ^ 2) - dt t q)
```

theorem1_upsilon Thm1Cert/Upsilon.lean

```
/-- **Theorem 1** with the Beigi–Rahimi-Keshari input in its Y form. -/
theorem theorem1_upsilon {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1)
    (hU : BRKUpsilon t (Thm2Cert.lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Thm2Cert.EE q ≤ ∑' n, negMulLog (c n ^ 2)
      + Thm2Cert.lam q t * ((G + 1 - t ^ 2) - Thm2Cert.dt t q)
```

theorem1_limit Thm1Cert/Corollaries.lean

```
/-- **Limit step.** If the affine bound holds along a sequence of states with `E_L → E` and
`D_L → D`, it holds in the limit. -/
theorem theorem1_limit {q t : ℝ} (E D : ℝ) (EL DL : ℕ → ℝ)
    (h : ∀ L, EE q ≤ EL L + lam q t * (DL L - dt t q))
    (hE : Tendsto EL atTop (𝓝 E)) (hD : Tendsto DL atTop (𝓝 D)) :
    EE q ≤ E + lam q t * (D - dt t q)
```

entropy_truncation Thm1Cert/Corollaries.lean

```
/-- Entropy of the normalized Schmidt truncations converges: with `P_M = ∑_{n ≤ M} c_n²`,
`H((c^{(M)})²) = S_M/P_M + ln P_M → H(c²)`. -/
theorem entropy_truncation (c : ℕ → ℝ) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2))) :
    Tendsto (fun M : ℕ => (∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) / (∑ n ∈ range (M + 1), c n ^ 2)
      + log (∑ n ∈ range (M + 1), c n ^ 2)) atTop (𝓝 (∑' n, negMulLog (c n ^ 2)))
```

theorem1_endpoint Thm1Cert/Assembly.lean

```
/-- **Eq. (S:endpoint)**: for `t < 1`, if the affine bound holds for all `q < t` and
`D_t(ψ) = d_t(t) = 1 − t²`, then `E(ψ) ≥ 𝓔(t)`. -/
theorem theorem1_endpoint (ht0 : 0 < t) (ht1 : t < 1) {E : ℝ}
    (h : ∀ q, 0 < q → q < t → EE q ≤ E + lam q t * (dt t t - dt t q)) : EE t ≤ E
```

theorem1_endpoint_state Thm1Cert/Upsilon.lean

```
/-- **Endpoint `q = t < 1`** (Eq. S:endpoint): a state with `D_t(ψ) = d_t(t) = 1 − t²`
(i.e. `G = 0`) has `E(ψ) ≥ 𝓔(t)`. -/
theorem theorem1_endpoint_state {t : ℝ} (ht0 : 0 < t) (ht1 : t < 1)
    (hU : ∀ q, 0 < q → q < t → BRKUpsilon t (Thm2Cert.lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) 0)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Thm2Cert.EE t ≤ ∑' n, negMulLog (c n ^ 2)
```

fixed_entanglement Thm1Cert/Assembly.lean

```
/-- **Eq. (S:fixedE)**: `E(ψ) ≤ 𝓔(q)` implies `D_t(ψ) ≥ d_t(q)`. -/
theorem fixed_entanglement (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) {E D : ℝ}
    (h1 : EE q ≤ E + lam q t * (D - dt t q)) (hE : E ≤ EE q) : dt t q ≤ D
```

fixed_entanglement_endpoint Thm1Cert/Corollaries.lean

```
/-- **Eq. (S:fixedE) at `q = t`**: `D_t(ψ) = G + 1 − t² ≥ 1 − t² = d_t(t)` since `G ≥ 0`. -/
theorem fixed_entanglement_endpoint {t G : ℝ} (ht0 : 0 < t) (ht1 : t < 1) (hG : 0 ≤ G) :
    dt t t ≤ G + 1 - t ^ 2
```

separability_criterion Thm1Cert/Corollaries.lean

```
/-- **Separability criterion (`q = 0` in Eq. S:fixedE)**: if the affine bound holds for all
`0 < q < t` and `E(ψ) ≤ 0` (a product state), then `D_t(ψ) ≥ 1 = d_t(0)`. -/
theorem separability_criterion {t E D : ℝ} (ht0 : 0 < t) (ht : t ≤ 1)
    (h : ∀ q, 0 < q → q < t → EE q ≤ E + lam q t * (D - dt t q)) (hE : E ≤ 0) : 1 ≤ D
```

7 Verification record

```
Lean toolchain: leanprover/lean4:v4.35.0-rc4
Mathlib commit: 49fd9dee36e7bbf345fe7981d2de6aeeb591f75b (8 Oct 2026)

== Clean build (lake build) ==
⚠ [2293/2307] Built Thm2Cert.MatrixLemma (11s)
⚠ [2294/2307] Built Thm2Cert.Reduction (4.7s)
⚠ [2296/2307] Built Thm1Cert.Defect (6.1s)
⚠ [2297/2307] Built Thm1Cert.Compression (6.6s)
⚠ [2298/2307] Built Thm1Cert.Closure (7.7s)
⚠ [2300/2307] Built Thm1Cert.Assembly (5.6s)
ℹ [2304/2307] Built Thm1Cert.Upsilon (2.4s)
ℹ [2305/2307] Built Thm1Cert.Corollaries (2.2s)
ℹ [2306/2307] Built Thm1Cert (1.7s)
Build completed successfully (2307 jobs).
real        0m49.849s
(only remaining warnings: Mathlib's generic 'consider adding module' notices)

== Axiom audit (lake env lean Certificate.lean) ==
'Thm1Cert.drop_lemma' depends on axioms: [propext, Classical.choice, Quot.sound]
```

```
'Thm1Cert.pointwise' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.defect_inequality_tsum' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.pava_exists' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.entropy_le_of_prefix' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.compression' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.H_thermal' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.F_thermal' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.support' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.Fmoments' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.brk_of_upsilon' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.thm1_spectrum' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.theorem1' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.theorem1_upsilon' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.theorem1_limit' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.entropy_truncation' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.theorem1_endpoint_state' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.fixed_entanglement' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.fixed_entanglement_endpoint' depends on axioms: [propext, Classical.choice, Quot.sound]
'Thm1Cert.separability_criterion' depends on axioms: [propext, Classical.choice, Quot.sound]

No sorry / admit / axiom declarations in Thm1Cert/*.lean, Thm2Cert/*.lean (3455 lines).

== SHA-256 ==
661c65779ee328d771311577f005439b775219c2e58092bc6e42a6701420de2a  Thm1Cert.lean
d6b7f8c68a8a86d6d10280e2e02621fd3950e03176343add15840101e1e8226a  Certificate.lean
fc5f133ca7b63b949bc8d302ed1c3921dae7659df8e1339c4ccea75db15a3fdf  Thm1Cert/Assembly.lean
665bfd8f35f90cefe67362b7e6ba7d9ccdb7434b165a3b8d5b2970cc0b9e93e7  Thm1Cert/Closure.lean
379302da07b64d2b10525243350e322d3f2b5b8014a61e4d4ae72bb27f1e4f4c  Thm1Cert/Compression.lean
eabc7cd2ab81821133023c7695e61ba54c57229934704dc985d98a8b4132a22a  Thm1Cert/Corollaries.lean
d91b7f411dbd614c300cd0203bda7f10f0c18668bddfeaa495fccd10c63b4a70  Thm1Cert/Defect.lean
99d3f7ccb3b53d9584c08c49d812f127e9ff4599566567ce94f6211117db185f  Thm1Cert/Upsilon.lean
d444f89ef18c12792308cf0e2d40fcb15fca44b22992042c76c5d156453f62a9  Thm2Cert/MatrixLemma.lean
74cb74257d090f4f2c87d2ecaa22a74e24589ce711643345a28d13713e0978ef  Thm2Cert/Reduction.lean
64f165e6350db97d4cea9f0067ba844de19005330fd4ec06c3b87764e56b5538  lakefile.toml
8f89aa44fccdb1a0b6cc768c7c9f54b94cd40e47a08d582f82d1235f13075cf7  lean-toolchain
```

Appendix: full source

Thm1Cert/Defect.lean

```
/-
Certificate for Theorem S1 (singular-value defect inequality) of the Supplemental Material of
G. Adesso, "Optimality of Gaussian entanglement of formation" (arXiv:2608.01909v3):

    G_t(C) ≥ ∑_n (n+1) d_n²,   d_n = (t c_n − c_{n+1})₊,

for decreasing Schmidt coefficients c_n ≥ 0 and 0 < t ≤ 1, in the Schmidt-basis matrix model:
    G_t(C) = ∑_{i,j} |t c_i X_ij − c_j Y_ij|²            (Eq. S:Gmatrix),
with the index identities ∑_j |X_ij|² − ∑_j |X_ji|² = 1 (and the same for Y) for every
index with c_i > 0 (Eq. S:index).

Certified: the positive-drop lemma (S:drop), the pointwise bound (S:pointwise) in both cases
i < j and i > j, the telescoping (S:defectN), and the limit N → ∞, for decreasing spectra
indexed by ℕ (finite or infinite support). Inputs: the matrix model (S:Gmatrix), i.e.
X = U†aU, Y = V†aV in paired Schmidt bases indexed by ℕ, and the index identities (S:index),
which follow from [a, a†] = 1. Paired bases indexed by ℕ exist for every state of finite Schmidt
rank (both kernels are infinite dimensional) and for infinite rank when the two kernels have equal
dimension; the remaining states are reached by the truncation limit (`Corollaries.lean`).
-/
import Mathlib.Analysis.SpecialFunctions.Log.Basic
import Mathlib.Analysis.Complex.Norm
import Mathlib.Topology.Algebra.InfiniteSum.Order
import Mathlib.Topology.Algebra.InfiniteSum.Real
import Thm2Cert.MatrixLemma

open Finset

set_option autoImplicit false

noncomputable section

namespace Thm1Cert

/-- `ite_t` / `ite_f` without deprecation warnings. -/
theorem ite_t {α : Sort*} {p : Prop} [Decidable p] (h : p) {a b : α} :
    (if p then a else b) = a := by simp [h]

theorem ite_f {α : Sort*} {p : Prop} [Decidable p] (h : ¬ p) {a b : α} :
    (if p then a else b) = b := by simp [h]

/-- Positive part. -/
def pos (x : ℝ) : ℝ := max x 0

lemma pos_nonneg (x : ℝ) : 0 ≤ pos x := le_max_right _ _

lemma le_pos (x : ℝ) : x ≤ pos x := le_max_left _ _

lemma pos_mul_self (x : ℝ) : x * pos x = pos x ^ 2 := by
  unfold pos
  rcases le_total x 0 with h | h
  · rw [max_eq_right h]; ring
  · rw [max_eq_left h]; ring

variable (t : ℝ) (c : ℕ → ℝ)

/-- The positive drops `d_n = (t c_n − c_{n+1})₊`. -/
def dd (n : ℕ) : ℝ := pos (t * c n - c (n + 1))

/-! ## Positive-drop lemma (S:drop) -/

/-- `D_ij = ∑_{k=i}^{j-1} (t c_k − c_{k+1})₊ ≤ (t c_i − c_j)₊` for `i ≤ j`. -/
theorem drop_lemma {t : ℝ} {c : ℕ → ℝ} (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (hc : Antitone c)
    (hc0 : ∀ n, 0 ≤ c n) {i j : ℕ} (hij : i ≤ j) :
    ∑ k ∈ Ico i j, dd t c k ≤ pos (t * c i - c j) := by
  induction j, hij using Nat.le_induction with
  | base =>
    simp only [Ico_self, sum_empty]; exact pos_nonneg _
  | succ j hij ih =>
    rw [sum_Ico_succ_top hij]
    have hcj : c j ≤ c i := hc hij
    have hcj1 : c (j + 1) ≤ c j := hc (Nat.le_succ j)
    have hc1 := hc0 (j + 1)
    unfold dd pos at *
    -- (t c_i − c_j)₊ + (t c_j − c_{j+1})₊ ≤ (t c_i − c_{j+1})₊
    have key : max (t * c i - c j) 0 + max (t * c j - c (j + 1)) 0
        ≤ max (t * c i - c (j + 1)) 0 := by
      have h1 : t * c j ≤ t * c i := mul_le_mul_of_nonneg_left hcj ht0
      have h2 : t * c j ≤ c j := by nlinarith [hc0 j]
      simp only [max_def]
      split_ifs <;> linarith
    linarith

/-! ## Pointwise bound (S:pointwise) -/

/-- Real form of the case `i < j`: for `0 ≤ D ≤ (A − B)₊`, `A, B ≥ 0`,
```

```
`(A a − B b)² − A D a² + B D b² ≥ 0`. -/
lemma real_case_lt {A B D : ℝ} (hA : 0 ≤ A) (hB : 0 ≤ B) (hD0 : 0 ≤ D) (hD : D ≤ pos (A - B))
    (a b : ℝ) : 0 ≤ (A * a - B * b) ^ 2 - A * D * a ^ 2 + B * D * b ^ 2 := by
  rcases hD0.eq_or_lt with h0 | hpos
  · subst h0; nlinarith [sq_nonneg (A * a - B * b)]
  · have hAB : D ≤ A - B := by
      unfold pos at hD
      rcases le_total (A - B) 0 with h | h
      · rw [max_eq_right h] at hD; linarith
      · rwa [max_eq_left h] at hD
    have e : (A * a - B * b) ^ 2 - A * D * a ^ 2 + B * D * b ^ 2
        = (A * (A - D)) * a ^ 2 - 2 * (A * B) * a * b + (B * (B + D)) * b ^ 2 := by ring
    rw [e]
    have := Thm2Cert.quad_nonneg (a := A * (A - D)) (c := B * (B + D)) (d := A * B)
      (mul_nonneg hA (by linarith)) (by nlinarith [mul_nonneg (mul_nonneg hA hB) hpos.le])
      (mul_nonneg hB (by linarith)) a b
    linarith

/-- Real form of the case `i > j`: for `0 ≤ D ≤ (t B − C)₊`, `A = t C`, `C ≤ B`, `t ≤ 1`,
`(A a − B b)² + A D a² − B D b² ≥ 0`. -/
lemma real_case_gt {t Cc B D : ℝ} (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (hC : 0 ≤ Cc) (hCB : Cc ≤ B)
    (hD0 : 0 ≤ D) (hD : D ≤ pos (t * B - Cc)) (a b : ℝ) :
    0 ≤ (t * Cc * a - B * b) ^ 2 + t * Cc * D * a ^ 2 - B * D * b ^ 2 := by
  have hB : 0 ≤ B := hC.trans hCB
  rcases hD0.eq_or_lt with h0 | hpos
  · subst h0; nlinarith [sq_nonneg (t * Cc * a - B * b)]
  · have hDB : D ≤ t * B - Cc := by
      unfold pos at hD
      rcases le_total (t * B - Cc) 0 with h | h
      · rw [max_eq_right h] at hD; linarith
      · rwa [max_eq_left h] at hD
    have hA : 0 ≤ t * Cc := mul_nonneg ht0 hC
    have hBD : D ≤ B := by nlinarith
    have hgap : 0 ≤ B - t * Cc - D := by nlinarith
    have e : (t * Cc * a - B * b) ^ 2 + t * Cc * D * a ^ 2 - B * D * b ^ 2
        = (t * Cc * (t * Cc + D)) * a ^ 2 - 2 * (t * Cc * B) * a * b + (B * (B - D)) * b ^ 2 := by
      ring
    rw [e]
    have := Thm2Cert.quad_nonneg (a := t * Cc * (t * Cc + D)) (c := B * (B - D))
      (d := t * Cc * B) (mul_nonneg hA (by linarith))
      (by nlinarith [mul_nonneg (mul_nonneg (mul_nonneg hA hB) hpos.le) hgap])
      (mul_nonneg hB (by linarith)) a b
    linarith

lemma norm_sq_ge (A B : ℝ) (hA : 0 ≤ A) (hB : 0 ≤ B) (x y : ℂ) :
    (A * ‖x‖ - B * ‖y‖) ^ 2 ≤ ‖(A : ℂ) * x - (B : ℂ) * y‖ ^ 2 := by
  have h := abs_norm_sub_norm_le ((A : ℂ) * x) ((B : ℂ) * y)
  rw [norm_mul, norm_mul, Complex.norm_real, Complex.norm_real, Real.norm_of_nonneg hA,
    Real.norm_of_nonneg hB] at h
  have h0 : 0 ≤ ‖(A : ℂ) * x - (B : ℂ) * y‖ := norm_nonneg _
  have := sq_le_sq' (abs_le.mp h).1 (abs_le.mp h).2
  exact this

/-! ## Dual potentials -/

/-- Truncated positive drops: `δ_k = d_k` for `k ≤ N`, `0` otherwise. -/
def dlt (N k : ℕ) : ℝ := if k ≤ N then dd t c k else 0

/-- `α_i = ∑_{k ≥ i} t c_k δ_k`. -/
def alpha (N i : ℕ) : ℝ := ∑ k ∈ Ico i (N + 1), t * c k * dlt t c N k

/-- `β_i = −∑_{k ≥ i} c_{k+1} δ_k`. -/
def beta (N i : ℕ) : ℝ := -∑ k ∈ Ico i (N + 1), c (k + 1) * dlt t c N k

lemma sum_Ico_trunc (N : ℕ) (g : ℕ → ℝ) (hg : ∀ k, N < k → g k = 0) {i j : ℕ} (hij : i ≤ j) :
    ∑ k ∈ Ico i (N + 1), g k - ∑ k ∈ Ico j (N + 1), g k = ∑ k ∈ Ico i j, g k := by
  rcases le_total j (N + 1) with hj | hj
  · rw [← sum_Ico_consecutive g hij hj]; ring
  · have e1 : ∑ k ∈ Ico j (N + 1), g k = 0 := by rw [Ico_eq_empty_of_le hj, sum_empty]
    rw [e1, sub_zero]
    rcases le_total i (N + 1) with hi | hi
    · rw [← sum_Ico_consecutive g hi hj]
      have : ∑ k ∈ Ico (N + 1) j, g k = 0 :=
        sum_eq_zero (fun k hk => hg k (by simp at hk; omega))
      rw [this, add_zero]
    · rw [Ico_eq_empty_of_le hi, sum_empty]
      exact (sum_eq_zero (fun k hk => hg k (by simp at hk; omega))).symm

variable {t c}

lemma dd_nonneg (k : ℕ) : 0 ≤ dd t c k := pos_nonneg _

lemma dlt_nonneg (N k : ℕ) : 0 ≤ dlt t c N k := by
  unfold dlt; split_ifs
  · exact dd_nonneg k
  · exact le_rfl

lemma dlt_le (N k : ℕ) : dlt t c N k ≤ dd t c k := by
  unfold dlt; split_ifs
  · exact le_rfl
  · exact dd_nonneg k
```

```

/-- **Eq. (S:pointwise)**: `|t c_i x − c_j y|² ≥ (α_i − α_j)|x|² + (β_i − β_j)|y|²`. -/
theorem pointwise (ht0 : 0 ≤ t) (ht1 : t ≤ 1) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n)
    (N i j : ℕ) (x y : ℂ) :
    (alpha t c N i - alpha t c N j) * ‖x‖ ^ 2 + (beta t c N i - beta t c N j) * ‖y‖ ^ 2
      ≤ ‖((t * c i : ℝ) : ℂ) * x - (c j : ℂ) * y‖ ^ 2 := by
  have hn := norm_sq_ge (t * c i) (c j) (mul_nonneg ht0 (hc0 i)) (hc0 j) x y
  have hg1 : ∀ k, N < k → t * c k * dlt t c N k = 0 := by
    intro k hk; unfold dlt; simp only [show ¬ k ≤ N by omega, ↓reduceIte]; ring
  have hg2 : ∀ k, N < k → c (k + 1) * dlt t c N k = 0 := by
    intro k hk; unfold dlt; simp only [show ¬ k ≤ N by omega, ↓reduceIte]; ring
  rcases lt_trichotomy i j with hij | hij | hij
  · -- i < j
    have hA := sum_Ico_trunc N _ hg1 hij.le
    have hB := sum_Ico_trunc N _ hg2 hij.le
    set D := ∑ k ∈ Ico i j, dlt t c N k
    have hD0 : 0 ≤ D := sum_nonneg (fun k _ => dlt_nonneg N k)
    have hD : D ≤ pos (t * c i - c j) :=
      (sum_le_sum (fun k _ => dlt_le N k)).trans (drop_lemma ht0 ht1 hc hc0 hij.le)
    have hP : ∑ k ∈ Ico i j, t * c k * dlt t c N k ≤ t * c i * D := by
      rw [mul_sum]
      exact sum_le_sum (fun k hk => by
        simp only [mem_Ico] at hk
        exact mul_le_mul_of_nonneg_right
          (mul_le_mul_of_nonneg_left (hc hk.1) ht0) (dlt_nonneg N k))
    have hQ : c j * D ≤ ∑ k ∈ Ico i j, c (k + 1) * dlt t c N k := by
      rw [mul_sum]
      exact sum_le_sum (fun k hk => by
        simp only [mem_Ico] at hk
        exact mul_le_mul_of_nonneg_right (hc (by omega)) (dlt_nonneg N k))
    have hreal := real_case_lt (mul_nonneg ht0 (hc0 i)) (hc0 j) hD0 hD ‖x‖ ‖y‖
    unfold alpha beta
    rw [hA, show -∑ k ∈ Ico i (N + 1), c (k + 1) * dlt t c N k
        - -∑ k ∈ Ico j (N + 1), c (k + 1) * dlt t c N k
        = -(∑ k ∈ Ico i (N + 1), c (k + 1) * dlt t c N k
          - ∑ k ∈ Ico j (N + 1), c (k + 1) * dlt t c N k) by ring, hB]
    have i1 := mul_le_mul_of_nonneg_right hP (sq_nonneg ‖x‖)
    have i2 := mul_le_mul_of_nonneg_right hQ (sq_nonneg ‖y‖)
    nlinarith
  · subst hij
    unfold alpha beta
    simp only [sub_self, zero_mul, add_zero]
    exact sq_nonneg _
  · -- i > j
    have hA := sum_Ico_trunc N _ hg1 hij.le
    have hB := sum_Ico_trunc N _ hg2 hij.le
    set D := ∑ k ∈ Ico j i, dlt t c N k
    have hD0 : 0 ≤ D := sum_nonneg (fun k _ => dlt_nonneg N k)
    have hD : D ≤ pos (t * c j - c i) :=
      (sum_le_sum (fun k _ => dlt_le N k)).trans (drop_lemma ht0 ht1 hc hc0 hij.le)
    have hP : t * c i * D ≤ ∑ k ∈ Ico j i, t * c k * dlt t c N k := by
      rw [mul_sum]
      exact sum_le_sum (fun k hk => by
        simp only [mem_Ico] at hk
        exact mul_le_mul_of_nonneg_right
          (mul_le_mul_of_nonneg_left (hc hk.2.le) ht0) (dlt_nonneg N k))
    have hQ : ∑ k ∈ Ico j i, c (k + 1) * dlt t c N k ≤ c j * D := by
      rw [mul_sum]
      exact sum_le_sum (fun k hk => by
        simp only [mem_Ico] at hk
        exact mul_le_mul_of_nonneg_right (hc (by omega)) (dlt_nonneg N k))
    have hreal := real_case_gt ht0 ht1 (hc0 i) (hc hij.le) hD0 hD ‖x‖ ‖y‖
    unfold alpha beta
    have eA : ∑ k ∈ Ico i (N + 1), t * c k * dlt t c N k
        - ∑ k ∈ Ico j (N + 1), t * c k * dlt t c N k
        = -∑ k ∈ Ico j i, t * c k * dlt t c N k := by linarith
    have eB : -∑ k ∈ Ico i (N + 1), c (k + 1) * dlt t c N k
        - -∑ k ∈ Ico j (N + 1), c (k + 1) * dlt t c N k
        = ∑ k ∈ Ico j i, c (k + 1) * dlt t c N k := by linarith
    rw [eA, eB]
    have i1 := mul_le_mul_of_nonneg_right hP (sq_nonneg ‖x‖)
    have i2 := mul_le_mul_of_nonneg_right hQ (sq_nonneg ‖y‖)
    nlinarith

/-! ## Telescoping (S:defectN) -/

lemma sum_alpha_beta (N : ℕ) :
    ∑ i ∈ range (N + 1), (alpha t c N i + beta t c N i)
      = ∑ k ∈ range (N + 1), ((k : ℝ) + 1) * dd t c k ^ 2 := by
  have h1 : ∀ i, alpha t c N i + beta t c N i
      = ∑ k ∈ Ico i (N + 1), (t * c k - c (k + 1)) * dlt t c N k := by
    intro i; unfold alpha beta; rw [← sub_eq_add_neg, ← sum_sub_distrib]
    exact sum_congr rfl (fun k _ => by ring)
  simp only [h1]
  rw [sum_comm' (t' := range (N + 1)) (s' := fun k => range (k + 1))
    (h := fun i k => by simp only [mem_range, mem_Ico]; omega)]
  refine sum_congr rfl (fun k hk => ?_)
  simp only [mem_range] at hk
  rw [sum_const, card_range, nsmul_eq_mul]
  unfold dlt; simp only [show k ≤ N by omega, ↓reduceIte]
  unfold dd
```

```
  rw [pos_mul_self]
  push_cast; ring

/-! ## Theorem S1 -/

/-- **Theorem S1** (singular-value defect inequality), matrix model.
`hG`: `G = ∑_{i,j} |t c_i X_ij − c_j Y_ij|²`; `hX`, `hY`: the index identities (S:index),
required only for indices with `c_i > 0`. Conclusion: `∑_{n ≤ N} (n+1) d_n² ≤ G` for every `N`,
hence `∑_n (n+1) d_n²` converges and is `≤ G`. -/
theorem defect_inequality (ht0 : 0 < t) (ht1 : t ≤ 1) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n)
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) (N : ℕ) :
    ∑ k ∈ range (N + 1), ((k : ℝ) + 1) * dd t c k ^ 2 ≤ G := by
  rw [← sum_alpha_beta]
  -- potentials vanish where c_i = 0
  have hα0 : ∀ i, c i = 0 → alpha t c N i = 0 := by
    intro i hi; unfold alpha
    exact sum_eq_zero (fun k hk => by
      simp only [mem_Ico] at hk
      have : c k = 0 := le_antisymm (hi ▸ hc hk.1) (hc0 k)
      rw [this]; ring)
  have hβ0 : ∀ i, c i = 0 → beta t c N i = 0 := by
    intro i hi; unfold beta
    rw [neg_eq_zero]
    exact sum_eq_zero (fun k hk => by
      simp only [mem_Ico] at hk
      have : c (k + 1) = 0 := le_antisymm (hi ▸ hc (by omega)) (hc0 _)
      rw [this]; ring)
  have hαN : ∀ i, N < i → alpha t c N i = 0 := by
    intro i hi; unfold alpha; rw [Ico_eq_empty_of_le (by omega), sum_empty]
  have hβN : ∀ i, N < i → beta t c N i = 0 := by
    intro i hi; unfold beta; rw [Ico_eq_empty_of_le (by omega), sum_empty, neg_zero]
  -- lower bound on the K × K box
  set R : ℕ → ℝ := fun K => ∑ i ∈ range (N + 1),
    (alpha t c N i * (∑ j ∈ range K, ‖X i j‖ ^ 2 - ∑ j ∈ range K, ‖X j i‖ ^ 2)
      + beta t c N i * (∑ j ∈ range K, ‖Y i j‖ ^ 2 - ∑ j ∈ range K, ‖Y j i‖ ^ 2)) with hR
  have hbox : ∀ K, N + 1 ≤ K → R K ≤ G := by
    intro K hK
    have h1 : ∑ ij ∈ range K ×ˢ range K, ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
        - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2 ≤ G :=
      sum_le_hasSum _ (fun _ _ => sq_nonneg _) hG
    have h2 : ∑ ij ∈ range K ×ˢ range K,
        ((alpha t c N ij.1 - alpha t c N ij.2) * ‖X ij.1 ij.2‖ ^ 2
          + (beta t c N ij.1 - beta t c N ij.2) * ‖Y ij.1 ij.2‖ ^ 2)
        ≤ ∑ ij ∈ range K ×ˢ range K, ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
          - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2 :=
      sum_le_sum (fun ij _ => pointwise ht0.le ht1 hc hc0 N ij.1 ij.2 _ _)
    have h3 : ∑ ij ∈ range K ×ˢ range K,
        ((alpha t c N ij.1 - alpha t c N ij.2) * ‖X ij.1 ij.2‖ ^ 2
          + (beta t c N ij.1 - beta t c N ij.2) * ‖Y ij.1 ij.2‖ ^ 2) = R K := by
      rw [sum_product]
      have hsplit : ∀ (P : ℕ → ℝ) (Z : ℕ → ℕ → ℂ), (∀ i, N < i → P i = 0) →
          ∑ i ∈ range K, ∑ j ∈ range K, (P i - P j) * ‖Z i j‖ ^ 2
            = ∑ i ∈ range (N + 1),
              P i * (∑ j ∈ range K, ‖Z i j‖ ^ 2 - ∑ j ∈ range K, ‖Z j i‖ ^ 2) := by
        intro P Z hP
        have e : ∑ i ∈ range K, ∑ j ∈ range K, (P i - P j) * ‖Z i j‖ ^ 2
            = ∑ i ∈ range K, P i * (∑ j ∈ range K, ‖Z i j‖ ^ 2 - ∑ j ∈ range K, ‖Z j i‖ ^ 2) := by
          simp only [sub_mul, sum_sub_distrib, mul_sub, mul_sum]
          rw [sum_comm (s := range K) (t := range K)
            (f := fun i j => P j * ‖Z i j‖ ^ 2)]
        rw [e, ← sum_range_add_sum_Ico _ hK]
        have : ∑ i ∈ Ico (N + 1) K,
            P i * (∑ j ∈ range K, ‖Z i j‖ ^ 2 - ∑ j ∈ range K, ‖Z j i‖ ^ 2) = 0 :=
          sum_eq_zero (fun i hi => by simp at hi; rw [hP i (by omega), zero_mul])
        rw [this, add_zero]
      have e2 : ∑ i ∈ range K, ∑ j ∈ range K,
          ((alpha t c N i - alpha t c N j) * ‖X i j‖ ^ 2
            + (beta t c N i - beta t c N j) * ‖Y i j‖ ^ 2)
          = ∑ i ∈ range K, ∑ j ∈ range K, (alpha t c N i - alpha t c N j) * ‖X i j‖ ^ 2
            + ∑ i ∈ range K, ∑ j ∈ range K, (beta t c N i - beta t c N j) * ‖Y i j‖ ^ 2 := by
        rw [← sum_add_distrib]; exact sum_congr rfl (fun i _ => sum_add_distrib)
      rw [e2, hsplit _ X hαN, hsplit _ Y hβN, hR, ← sum_add_distrib]
    linarith
  -- limit K → ∞
  have hlim : Filter.Tendsto R Filter.atTop
      (nhds (∑ i ∈ range (N + 1), (alpha t c N i + beta t c N i))) := by
    rw [hR]
    have : ∑ i ∈ range (N + 1), (alpha t c N i + beta t c N i)
        = ∑ i ∈ range (N + 1), (alpha t c N i * (rX i - cX i)
          + beta t c N i * (rY i - cY i)) := by
      refine sum_congr rfl (fun i _ => ?_)
      rcases (hc0 i).eq_or_lt with h0 | hpos
      · rw [hα0 i h0.symm, hβ0 i h0.symm]; ring
```

```
      · rw [hX i hpos, hY i hpos]; ring
    rw [this]
    refine tendsto_finsetSum _ (fun i _ => ?_)
    rcases (hc0 i).eq_or_lt with h0 | hpos
    · simp only [hα0 i h0.symm, hβ0 i h0.symm, zero_mul, add_zero]
      exact tendsto_const_nhds
    · refine Filter.Tendsto.add (Filter.Tendsto.const_mul _ ?_) (Filter.Tendsto.const_mul _ ?_)
      · exact ((hrX i hpos).tendsto_sum_nat).sub ((hcX i hpos).tendsto_sum_nat)
      · exact ((hrY i hpos).tendsto_sum_nat).sub ((hcY i hpos).tendsto_sum_nat)
  exact le_of_tendsto hlim (Filter.eventually_atTop.mpr ⟨N + 1, hbox⟩)

/-- Consequence: the defect series converges and `∑_n (n+1) d_n² ≤ G_t(C)`. -/
theorem defect_inequality_tsum (ht0 : 0 < t) (ht1 : t ≤ 1) (hc : Antitone c)
    (hc0 : ∀ n, 0 ≤ c n) (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Summable (fun n : ℕ => ((n : ℝ) + 1) * dd t c n ^ 2) ∧
    ∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2 ≤ G := by
  have hnn : ∀ n : ℕ, 0 ≤ ((n : ℝ) + 1) * dd t c n ^ 2 := fun n => by positivity
  have hb : ∀ n, ∑ i ∈ range n, ((i : ℝ) + 1) * dd t c i ^ 2 ≤ G := by
    intro n
    rcases n with _ | n
    · simp only [range_zero, sum_empty]
      exact hG.nonneg (fun _ => sq_nonneg _)
    · exact defect_inequality ht0 ht1 hc hc0 X Y G hG rX cX rY cY hrX hcX hrY hcY hX hY n
  exact ⟨summable_of_sum_range_le hnn hb, Real.tsum_le_of_sum_range_le hnn hb⟩

end Thm1Cert
```

Thm1Cert/Compression.lean

```
/-
Certificate for Theorem S2 (geometric isotonic compression) of the Supplemental Material of
G. Adesso, "Optimality of Gaussian entanglement of formation" (arXiv:2608.01909v3).

For c_0 ≥ … ≥ c_M > 0 with ∑ c_n² = 1 (c_{M+1} = 0) and t > 0 there is a normalized
z_0, …, z_M ≥ 0 (z_{M+1} = 0) with
    z_{n+1} ≤ t z_n,     H(z²) ≤ H(c²),
    ∑ (n+1)(t z_n − z_{n+1})² ≤ ∑ (n+1)(t c_n − c_{n+1})₊².

The pool-adjacent-violators construction is formalized as an existence proof by strong induction
on the number of level breaks: pooling two adjacent maximal constant runs that violate
monotonicity preserves the total mass, can only raise every prefix sum, and does not increase
the drop energy (S:senergy); prefix domination then gives H(z²) ≤ H(c²) by a tangent-line /
Abel-summation form of Schur concavity.
-/
import Mathlib.Analysis.SpecialFunctions.Log.NegMulLog
import Mathlib.Analysis.SpecialFunctions.Sqrt
import Thm1Cert.Defect

open Finset Real

set_option autoImplicit false

noncomputable section

namespace Thm1Cert

variable (M : ℕ) (t : ℝ)

/-- Weights `w_n = t^{2n}`. -/
def wt (n : ℕ) : ℝ := (t ^ n) ^ 2

/-- Total weighted mass `∑_{n ≤ M} w_n ℓ_n`. -/
def mass (ℓ : ℕ → ℝ) : ℝ := ∑ n ∈ range (M + 1), wt t n * ℓ n

/-- Prefix mass `∑_{n ≤ K} w_n ℓ_n`. -/
def pre (ℓ : ℕ → ℝ) (K : ℕ) : ℝ := ∑ n ∈ range (K + 1), wt t n * ℓ n

/-- Drop energy (S:senergy): `∑_{n ≤ M} (n+1) t^{2n+2} ((√ℓ_n − √ℓ_{n+1})₊)²`. -/
def en (ℓ : ℕ → ℝ) : ℝ :=
  ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t ^ (n + 1)) ^ 2 * pos (√(ℓ n) - √(ℓ (n + 1))) ^ 2

/-- Number of level breaks on `[0, M]`. -/
def brk (ℓ : ℕ → ℝ) : ℕ := ((range M).filter (fun n => ℓ n ≠ ℓ (n + 1))).card

/-- Invariants of the pooling procedure, relative to the initial levels `r`. -/
structure Inv (r ℓ : ℕ → ℝ) : Prop where
  nonneg : ∀ n, 0 ≤ ℓ n
  supp : ∀ n, M < n → ℓ n = 0
  mass_eq : mass M t ℓ = mass M t r
  prefix_ge : ∀ K, K ≤ M → pre t r K ≤ pre t ℓ K
  en_le : en M t ℓ ≤ en M t r
```

```

variable {M t}

lemma wt_pos (ht : 0 < t) (n : ℕ) : 0 < wt t n := by unfold wt; positivity

lemma pos_mul_left {k : ℝ} (hk : 0 ≤ k) (u : ℝ) : pos (k * u) = k * pos u := by
  unfold pos
  rcases le_total u 0 with h | h
  · rw [max_eq_right h, max_eq_right (mul_nonpos_of_nonneg_of_nonpos hk h), mul_zero]
  · rw [max_eq_left h, max_eq_left (mul_nonneg hk h)]

/-- **Pooling step.** If two adjacent levels violate monotonicity, pooling the two maximal
constant runs around them preserves the invariants and strictly reduces the number of breaks. -/
theorem pool_step (ht : 0 < t) {r ℓ : ℕ → ℝ} (hI : Inv M t r ℓ) {n : ℕ} (hn : n < M)
    (hv : ℓ n < ℓ (n + 1)) : ∃ ℓ', Inv M t r ℓ' ∧ brk M ℓ' < brk M ℓ := by
  classical
  -- left run [a, n]
  have hPa : ∃ a, ∀ k, a ≤ k → k ≤ n → ℓ k = ℓ n :=
    ⟨n, fun k h1 h2 => by rw [le_antisymm h2 h1]⟩
  set a := Nat.find hPa with ha_def
  have ha : ∀ k, a ≤ k → k ≤ n → ℓ k = ℓ n := Nat.find_spec hPa
  have han : a ≤ n := Nat.find_min' hPa (fun k h1 h2 => by rw [le_antisymm h2 h1])
  have hamin : 0 < a → ℓ (a - 1) ≠ ℓ n := by
    intro hpos heq
    apply Nat.find_min hPa (Nat.sub_lt hpos one_pos)
    intro k h1 h2
    rcases Nat.lt_or_ge k a with hk | hk
    · rw [show k = a - 1 by omega]; exact heq
    · exact ha k hk h2
  -- right run [n+1, b]
  set Pb : ℕ → Prop := fun b => n + 1 ≤ b ∧ ∀ k, n + 1 ≤ k → k ≤ b → ℓ k = ℓ (n + 1) with hPb
  have hPb1 : Pb (n + 1) := ⟨le_rfl, fun k h1 h2 => by rw [le_antisymm h2 h1]⟩
  set b := Nat.findGreatest Pb M with hb_def
  have hb : Pb b := Nat.findGreatest_spec (m := n + 1) (by omega) hPb1
  have hbM : b ≤ M := Nat.findGreatest_le M
  have hbmax : b < M → ℓ (b + 1) ≠ ℓ (n + 1) := by
    intro hlt heq
    apply Nat.findGreatest_is_greatest (P := Pb) (k := b + 1) (Nat.lt_succ_self b) (show b + 1 ≤ M from hlt)
    refine ⟨by omega, fun k h1 h2 => ?_⟩
    rcases Nat.lt_or_ge b k with hk | hk
    · rw [show k = b + 1 by omega]; exact heq
    · exact hb.2 k h1 hk
  have hnb : n + 1 ≤ b := hb.1
  set x := ℓ n with hx
  set y := ℓ (n + 1) with hy
  have hval : ∀ k, a ≤ k → k ≤ b → x ≤ ℓ k ∧ ℓ k ≤ y := by
    intro k h1 h2
    rcases Nat.lt_or_ge n k with hk | hk
    · rw [hb.2 k hk h2]; exact ⟨hv.le, le_rfl⟩
    · rw [ha k h1 hk]; exact ⟨le_rfl, hv.le⟩
  -- pooled level
  set W := ∑ k ∈ Icc a b, wt t k with hW
  have hWpos : 0 < W := sum_pos (fun k _ => wt_pos ht k) ⟨a, by simp; omega⟩
  set m := (∑ k ∈ Icc a b, wt t k * ℓ k) / W with hm
  have hxm : x ≤ m := by
    rw [hm, le_div_iff₀ hWpos, hW, mul_sum]
    exact sum_le_sum (fun k hk => by
      simp only [mem_Icc] at hk
      rw [mul_comm]; exact mul_le_mul_of_nonneg_left (hval k hk.1 hk.2).1 (wt_pos ht k).le)
  have hmy : m ≤ y := by
    rw [hm, div_le_iff₀ hWpos, hW, mul_sum]
    exact sum_le_sum (fun k hk => by
      simp only [mem_Icc] at hk
      rw [mul_comm y]; exact mul_le_mul_of_nonneg_left (hval k hk.1 hk.2).2 (wt_pos ht k).le)
  set ℓ' : ℕ → ℝ := fun k => if a ≤ k ∧ k ≤ b then m else ℓ k with hℓ'
  set g : ℕ → ℝ := fun k => wt t k * (ℓ' k - ℓ k) with hg
  have hgout : ∀ k, ¬ (a ≤ k ∧ k ≤ b) → g k = 0 := by
    intro k hk; simp only [hg, hℓ', ite_f hk, sub_self, mul_zero]
  have hgtot : ∑ k ∈ range (M + 1), g k = 0 := by
    have hsub : Icc a b ⊆ range (M + 1) := by
      intro k hk; simp only [mem_Icc] at hk; simp only [mem_range]; omega
    rw [← sum_subset hsub (fun k _ hk => hgout k (by simpa using hk))]
    have : ∑ k ∈ Icc a b, g k = m * W - ∑ k ∈ Icc a b, wt t k * ℓ k := by
      rw [hW, mul_sum, ← sum_sub_distrib]
      refine sum_congr rfl (fun k hk => ?_)
      simp only [mem_Icc] at hk
      simp only [hg, hℓ', ite_t hk]; ring
    rw [this, hm, div_mul_cancel₀ _ hWpos.ne', sub_self]
  have hgnn : ∀ k, k ≤ n → 0 ≤ g k := by
    intro k hk
    by_cases h : a ≤ k ∧ k ≤ b
    · simp only [hg, hℓ', ite_t h, ha k h.1 hk]
      exact mul_nonneg (wt_pos ht k).le (by linarith)
    · rw [hgout k h]
  have hgnp : ∀ k, n + 1 ≤ k → g k ≤ 0 := by
    intro k hk
    by_cases h : a ≤ k ∧ k ≤ b
    · simp only [hg, hℓ', ite_t h, hb.2 k hk h.2]
      exact mul_nonpos_of_nonneg_of_nonpos (wt_pos ht k).le (by linarith)
    · rw [hgout k h]
  have hpre_diff : ∀ K, pre t ℓ' K - pre t ℓ K = ∑ k ∈ range (K + 1), g k := by
```

```
    intro K; unfold pre; rw [← sum_sub_distrib]
    exact sum_congr rfl (fun k _ => by simp only [hg]; ring)
  refine ⟨ℓ', ⟨?_, ?_, ?_, ?_, ?_⟩, ?_⟩
  · -- nonneg
    intro k
    simp only [hℓ']
    split_ifs
    · exact (hI.nonneg n).trans hxm
    · exact hI.nonneg k
  · -- support
    intro k hk
    simp only [hℓ']
    rw [ite_f (by omega)]
    exact hI.supp k hk
  · -- mass
    have h1 := hpre_diff M
    have h2 := hI.mass_eq
    unfold pre at h1
    unfold mass at h2 ⊢
    linarith
  · -- prefix
    intro K hK
    have hdiff : 0 ≤ ∑ k ∈ range (K + 1), g k := by
      rcases Nat.lt_or_ge n K with hnK | hnK
      · have hsplit := sum_range_add_sum_Ico g (show K + 1 ≤ M + 1 by omega)
        have : ∑ k ∈ Ico (K + 1) (M + 1), g k ≤ 0 :=
          sum_nonpos (fun k hk => by simp at hk; exact hgnp k (by omega))
        linarith
      · exact sum_nonneg (fun k hk => by simp at hk; exact hgnn k (by omega))
    have := hpre_diff K
    linarith [hI.prefix_ge K hK]
  · -- energy
    refine le_trans (sum_le_sum (fun j _ => ?_)) hI.en_le
    apply mul_le_mul_of_nonneg_left _ (by positivity)
    apply pow_le_pow_left₀ (pos_nonneg _)
    by_cases hj : a ≤ j ∧ j ≤ b
    · by_cases hj1 : a ≤ j + 1 ∧ j + 1 ≤ b
      · simp only [hℓ', ite_t hj, ite_t hj1, sub_self]
        unfold pos; rw [max_self]; exact le_max_right _ _
      · have hjb : j = b := by omega
        have hyj : ℓ j = y := by rw [hjb]; exact hb.2 b hnb le_rfl
        simp only [hℓ', ite_t hj, ite_f hj1]
        have : √m ≤ √(ℓ j) := Real.sqrt_le_sqrt (by rw [hyj]; exact hmy)
        exact max_le_max (by linarith) le_rfl
    · by_cases hj1 : a ≤ j + 1 ∧ j + 1 ≤ b
      · have hja : j + 1 = a := by omega
        have hxa : ℓ (j + 1) = x := by rw [hja]; exact ha a le_rfl han
        simp only [hℓ', ite_f hj, ite_t hj1]
        have : √(ℓ (j + 1)) ≤ √m := Real.sqrt_le_sqrt (by rw [hxa]; exact hxm)
        exact max_le_max (by linarith) le_rfl
      · simp only [hℓ', ite_f hj, ite_f hj1]; exact le_rfl
  · -- breaks
    unfold brk
    have hnS : n ∈ (range M).filter (fun j => ℓ j ≠ ℓ (j + 1)) := by
      simp only [mem_filter, mem_range]; exact ⟨hn, hv.ne⟩
    have hsub : (range M).filter (fun j => ℓ' j ≠ ℓ' (j + 1))
        ⊆ ((range M).filter (fun j => ℓ j ≠ ℓ (j + 1))).erase n := by
      intro j hjS
      simp only [mem_filter, mem_range] at hjS
      obtain ⟨hjM, hne⟩ := hjS
      simp only [mem_erase, mem_filter, mem_range]
      by_cases hj : a ≤ j ∧ j ≤ b
      · by_cases hj1 : a ≤ j + 1 ∧ j + 1 ≤ b
        · simp only [hℓ', ite_t hj, ite_t hj1] at hne; exact absurd rfl hne
        · have hjb : j = b := by omega
          refine ⟨by omega, hjM, ?_⟩
          rw [hjb, hb.2 b hnb le_rfl]
          exact fun h => hbmax (by omega) h.symm
      · by_cases hj1 : a ≤ j + 1 ∧ j + 1 ≤ b
        · have hja : j + 1 = a := by omega
          refine ⟨by omega, hjM, ?_⟩
          rw [hja, ha a le_rfl han, show j = a - 1 by omega]
          exact hamin (by omega)
        · simp only [hℓ', ite_f hj, ite_f hj1] at hne
          exact ⟨fun h => hj ⟨by omega, by omega⟩, hjM, hne⟩
    calc _ ≤ (((range M).filter (fun j => ℓ j ≠ ℓ (j + 1))).erase n).card := card_le_card hsub
      _ < _ := card_erase_lt_of_mem hnS

/-- **Pool-adjacent-violators, existence form.** There are levels satisfying the invariants
that are nonincreasing on `[0, M]`. -/
theorem pava_exists (ht : 0 < t) {r : ℕ → ℝ} (hI : Inv M t r r) :
    ∃ ℓ, Inv M t r ℓ ∧ ∀ n, n < M → ℓ (n + 1) ≤ ℓ n := by
  suffices h : ∀ k ℓ, brk M ℓ = k → Inv M t r ℓ →
      ∃ ℓ', Inv M t r ℓ' ∧ ∀ n, n < M → ℓ' (n + 1) ≤ ℓ' n from h _ _ rfl hI
  intro k
  induction k using Nat.strong_induction_on with
  | _ k ih =>
    intro ℓ hk hIℓ
    by_cases hall : ∀ n, n < M → ℓ (n + 1) ≤ ℓ n
    · exact ⟨ℓ, hIℓ, hall⟩
    · push Not at hall
```

```
      obtain ⟨n, hn, hv⟩ := hall
      obtain ⟨ℓ', hI', hlt⟩ := pool_step ht hIℓ hn hv
      exact ih _ (hk ▸ hlt) ℓ' rfl hI'

/-! ## Schur concavity of the entropy under prefix domination -/

lemma tangent (p q : ℝ) (hp : 0 < p) (hq : 0 ≤ q) :
    negMulLog q ≤ negMulLog p + (-log p - 1) * (q - p) := by
  unfold negMulLog
  rcases hq.eq_or_lt with h0 | hqp
  · subst h0; simp; nlinarith
  · have h := Real.log_le_sub_one_of_pos (div_pos hp hqp)
    rw [Real.log_div hp.ne' hqp.ne'] at h
    have : q * (log p - log q) ≤ q * (p / q - 1) := mul_le_mul_of_nonneg_left h hqp.le
    have e : q * (p / q - 1) = p - q := by field_simp
    rw [e] at this
    nlinarith

/-- If `p` is positive and nonincreasing on `[0, M]` and every prefix sum of `q ≥ 0` dominates
that of `p`, with equal totals, then `H(q) ≤ H(p)`. -/
theorem entropy_le_of_prefix (p q : ℕ → ℝ) (hp : ∀ n, n ≤ M → 0 < p n)
    (hanti : ∀ n, n < M → p (n + 1) ≤ p n) (hq : ∀ n, 0 ≤ q n)
    (hpre : ∀ K, K ≤ M → ∑ n ∈ range (K + 1), p n ≤ ∑ n ∈ range (K + 1), q n)
    (htot : ∑ n ∈ range (M + 1), p n = ∑ n ∈ range (M + 1), q n) :
    ∑ n ∈ range (M + 1), negMulLog (q n) ≤ ∑ n ∈ range (M + 1), negMulLog (p n) := by
  set g : ℕ → ℝ := fun n => -log (p n) - 1 with hg
  have hgmono : ∀ n, n < M → g n ≤ g (n + 1) := by
    intro n hn
    simp only [hg]
    have := Real.log_le_log (hp (n + 1) (by omega)) (hanti n hn)
    linarith
  -- Abel summation: ∑_{n ≤ K} g_n Δ_n ≤ g_K S_K
  set S : ℕ → ℝ := fun K => ∑ n ∈ range (K + 1), (q n - p n) with hS
  have hSnn : ∀ K, K ≤ M → 0 ≤ S K := by
    intro K hK; simp only [hS, sum_sub_distrib]; linarith [hpre K hK]
  have habel : ∀ K, K ≤ M → ∑ n ∈ range (K + 1), g n * (q n - p n) ≤ g K * S K := by
    intro K
    induction K with
    | zero => intro _; simp [hS]
    | succ K ih =>
      intro hK
      rw [sum_range_succ, show S (K + 1) = S K + (q (K + 1) - p (K + 1)) by
        simp only [hS]; rw [sum_range_succ]]
      have h1 := ih (by omega)
      have h2 := mul_le_mul_of_nonneg_right (hgmono K (by omega)) (hSnn K (by omega))
      nlinarith
  have hSM : S M = 0 := by simp only [hS, sum_sub_distrib]; linarith
  have hlin : ∑ n ∈ range (M + 1), g n * (q n - p n) ≤ 0 := by
    have := habel M le_rfl; rw [hSM, mul_zero] at this; exact this
  have htan : ∑ n ∈ range (M + 1), negMulLog (q n)
      ≤ ∑ n ∈ range (M + 1), (negMulLog (p n) + g n * (q n - p n)) :=
    sum_le_sum (fun n hn => by
      simp only [mem_range] at hn
      exact tangent (p n) (q n) (hp n (by omega)) (hq n))
  rw [sum_add_distrib] at htan
  linarith

/-! ## Theorem S2 -/

/-- **Theorem S2** (geometric isotonic compression). -/
theorem compression (ht : 0 < t) (c : ℕ → ℝ) (hcpos : ∀ n, n ≤ M → 0 < c n)
    (hanti : ∀ n, n < M → c (n + 1) ≤ c n) (hsupp : ∀ n, M < n → c n = 0)
    (hnorm : ∑ n ∈ range (M + 1), c n ^ 2 = 1) :
    ∃ z : ℕ → ℝ, (∀ n, 0 ≤ z n) ∧ (∀ n, M < n → z n = 0) ∧
      ∑ n ∈ range (M + 1), z n ^ 2 = 1 ∧ (∀ n, z (n + 1) ≤ t * z n) ∧
      ∑ n ∈ range (M + 1), negMulLog (z n ^ 2) ≤ ∑ n ∈ range (M + 1), negMulLog (c n ^ 2) ∧
      ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2
        ≤ ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * pos (t * c n - c (n + 1)) ^ 2 := by
  classical
  set r : ℕ → ℝ := fun n => if n ≤ M then c n ^ 2 / wt t n else 0 with hr
  have hrnn : ∀ n, 0 ≤ r n := by
    intro n; simp only [hr]; split_ifs
    · exact div_nonneg (sq_nonneg _) (wt_pos ht n).le
    · exact le_rfl
  -- c_n = t^n √r_n for all n ≤ M + 1
  have hcr : ∀ n, n ≤ M + 1 → c n = t ^ n * √(r n) := by
    intro n hn
    rcases Nat.lt_or_ge M n with h | h
    · rw [hsupp n h]; simp only [hr, ite_f (by omega : ¬ n ≤ M), Real.sqrt_zero, mul_zero]
    · simp only [hr, ite_t h]
      rw [Real.sqrt_div (sq_nonneg _), Real.sqrt_sq (hcpos n h).le, wt,
        Real.sqrt_sq (by positivity)]
      field_simp
  have hwr : ∀ n, n ≤ M → wt t n * r n = c n ^ 2 := by
    intro n hn; simp only [hr, ite_t hn]; field_simp [(wt_pos ht n).ne']
  have hIr : Inv M t r r := ⟨hrnn, fun n hn => by simp only [hr, show ¬ n ≤ M by omega, ↓reduceIte],
    rfl, fun _ _ => le_rfl, le_rfl⟩
  obtain ⟨ℓ, hI, hmono⟩ := pava_exists ht hIr
  refine ⟨fun n => t ^ n * √(ℓ n), fun n => by positivity, fun n hn => by
    simp only [hI.supp n hn, Real.sqrt_zero, mul_zero], ?_, ?_, ?_, ?_⟩
  · -- normalization
```

```
    have : ∑ n ∈ range (M + 1), (t ^ n * √(ℓ n)) ^ 2 = mass M t ℓ := by
      unfold mass wt
      exact sum_congr rfl (fun n _ => by rw [mul_pow, Real.sq_sqrt (hI.nonneg n)])
    rw [this, hI.mass_eq]
    unfold mass
    rw [← hnorm]
    exact sum_congr rfl (fun n hn => hwr n (by simp at hn; omega))
  · -- ratio
    intro n
    show t ^ (n + 1) * √(ℓ (n + 1)) ≤ t * (t ^ n * √(ℓ n))
    rcases Nat.lt_or_ge n M with hn | hn
    · rw [pow_succ]
      have := Real.sqrt_le_sqrt (hmono n hn)
      have ht' : 0 ≤ t ^ n * t := by positivity
      calc t ^ n * t * √(ℓ (n + 1)) ≤ t ^ n * t * √(ℓ n) := mul_le_mul_of_nonneg_left this ht'
        _ = t * (t ^ n * √(ℓ n)) := by ring
    · simp only [hI.supp (n + 1) (by omega), Real.sqrt_zero, mul_zero]
      positivity
  · -- entropy
    have hq : ∀ n, (t ^ n * √(ℓ n)) ^ 2 = wt t n * ℓ n := by
      intro n; unfold wt; rw [mul_pow, Real.sq_sqrt (hI.nonneg n)]
    simp only [hq]
    apply entropy_le_of_prefix (fun n => c n ^ 2) (fun n => wt t n * ℓ n)
    · intro n hn; exact pow_pos (hcpos n hn) 2
    · intro n hn
      exact pow_le_pow_left₀ (hcpos (n + 1) (by omega)).le (hanti n hn) 2
    · intro n; exact mul_nonneg (wt_pos ht n).le (hI.nonneg n)
    · intro K hK
      have := hI.prefix_ge K hK
      unfold pre at this
      calc ∑ n ∈ range (K + 1), c n ^ 2 = ∑ n ∈ range (K + 1), wt t n * r n :=
            sum_congr rfl (fun n hn => (hwr n (by simp at hn; omega)).symm)
        _ ≤ _ := this
    · have := hI.mass_eq
      unfold mass at this
      rw [this]
      exact sum_congr rfl (fun n hn => (hwr n (by simp at hn; omega)).symm)
  · -- energy
    have hL : ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t * (t ^ n * √(ℓ n))
        - t ^ (n + 1) * √(ℓ (n + 1))) ^ 2 = en M t ℓ := by
      unfold en
      refine sum_congr rfl (fun n hn => ?_)
      simp only [mem_range] at hn
      have hd : 0 ≤ √(ℓ n) - √(ℓ (n + 1)) := by
        rcases Nat.lt_or_ge n M with h | h
        · have := Real.sqrt_le_sqrt (hmono n h); linarith
        · rw [hI.supp (n + 1) (by omega), Real.sqrt_zero, sub_zero]; positivity
      rw [show pos (√(ℓ n) - √(ℓ (n + 1))) = √(ℓ n) - √(ℓ (n + 1)) from max_eq_left hd]
      ring
    have hR : ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * pos (t * c n - c (n + 1)) ^ 2 = en M t r := by
      unfold en
      refine sum_congr rfl (fun n hn => ?_)
      simp only [mem_range] at hn
      rw [hcr n (by omega), hcr (n + 1) (by omega)]
      have : t * (t ^ n * √(r n)) - t ^ (n + 1) * √(r (n + 1))
          = t ^ (n + 1) * (√(r n) - √(r (n + 1))) := by ring
      rw [this]
      rw [pos_mul_left (by positivity : (0:ℝ) ≤ t ^ (n + 1))]
      ring
    rw [hL, hR]
    exact hI.en_le

end Thm1Cert
```

Thm1Cert/Closure.lean

```
/-
Certificate for the thermal log-Sobolev closure of the Supplemental Material of
G. Adesso, "Optimality of Gaussian entanglement of formation" (arXiv:2608.01909v3):

  * Eq. (S:thermalF): for z^{(s)}_n = √(1−s²) sⁿ,
        H((z^{(s)})²) = 𝓔(s),     𝓕_t(z^{(s)}) = (t−s)²/(1−s²);
  * Eq. (S:support): for 0 < q < t ≤ 1 and λ = λ_{q,t}, the function
        φ(s) = 𝓔(s) + λ (t−s)²/(1−s²)
    attains its minimum over 0 ≤ s < 1 at s = q (monotonicity of Λ_t, Eq. S:LambdaMon).

The meta log-Sobolev theorem of Beigi and Rahimi-Keshari is NOT formalized; it enters
`Assembly.lean` as an explicit hypothesis (`BRK`).
-/
import Mathlib.Analysis.SpecialFunctions.Log.NegMulLog
import Mathlib.Analysis.SpecificLimits.Normed
import Mathlib.Analysis.Calculus.Deriv.MeanValue
import Thm1Cert.Compression
import Thm2Cert.Reduction

open Finset Real

set_option autoImplicit false

```

```
noncomputable section

namespace Thm1Cert

open Thm2Cert (EE gfun lam dt EE_formula lam_pos)

/-- Entropy `H(z²) = ∑ −z_n² ln z_n²` of a square-root sequence. -/
def Hs (z : ℕ → ℝ) : ℝ := ∑' n, negMulLog (z n ^ 2)

/-- `𝓕_t(z) = ∑ (n+1)(t z_n − z_{n+1})²` (Eq. S:Fdef). -/
def Ft (t : ℝ) (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2

/-- Thermal square-root sequence `z^{(s)}_n = √(1−s²) sⁿ` (Eq. S:thermalz). -/
def zs (s : ℝ) (n : ℕ) : ℝ := √(1 - s ^ 2) * s ^ n

lemma zs_sq {s : ℝ} (hs1 : s < 1) (hs0 : 0 ≤ s) (n : ℕ) :
    zs s n ^ 2 = (1 - s ^ 2) * (s ^ 2) ^ n := by
  unfold zs
  rw [mul_pow, Real.sq_sqrt (by nlinarith), ← pow_mul, ← pow_mul, mul_comm 2 n]

lemma EE_zero : EE 0 = 0 := by
  unfold EE gfun; simp

/-- **Eq. (S:thermalF), entropy part**: `H((z^{(s)})²) = 𝓔(s)`. -/
theorem H_thermal {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) : Hs (zs s) = EE s := by
  rcases hs0.eq_or_lt with h0 | hpos
  · subst h0
    rw [EE_zero]
    unfold Hs
    have : ∀ n, negMulLog (zs 0 n ^ 2) = 0 := by
      intro n
      rw [zs_sq (by norm_num) le_rfl]
      rcases n with _ | n
      · simp
      · simp
    simp only [this, tsum_zero]
  · have hx0 : 0 ≤ s ^ 2 := sq_nonneg s
    have hx1 : s ^ 2 < 1 := by nlinarith
    have ha : 0 < 1 - s ^ 2 := by linarith
    have hg := hasSum_geometric_of_lt_one hx0 hx1
    have hn := hasSum_coe_mul_geometric_of_norm_lt_one
      (show ‖s ^ 2‖ < 1 by rw [Real.norm_of_nonneg hx0]; exact hx1)
    have hsum := (hg.mul_left (-(1 - s ^ 2) * log (1 - s ^ 2))).add
      (hn.mul_left (-(1 - s ^ 2) * log (s ^ 2)))
    unfold Hs
    have hterm : ∀ n : ℕ, negMulLog (zs s n ^ 2)
        = -(1 - s ^ 2) * log (1 - s ^ 2) * (s ^ 2) ^ n
          + -(1 - s ^ 2) * log (s ^ 2) * ((n : ℝ) * (s ^ 2) ^ n) := by
      intro n
      rw [zs_sq hs1 hs0 n]
      unfold negMulLog
      rw [Real.log_mul ha.ne' (pow_ne_zero _ (by positivity)), Real.log_pow]
      ring
    simp only [hterm]
    rw [hsum.tsum_eq, EE_formula hpos hs1, Real.log_pow]
    field_simp
    push_cast
    ring

/-- **Eq. (S:thermalF), energy part**: `𝓕_t(z^{(s)}) = (t−s)²/(1−s²)`. -/
theorem F_thermal (t : ℝ) {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) :
    Ft t (zs s) = (t - s) ^ 2 / (1 - s ^ 2) := by
  have hx0 : 0 ≤ s ^ 2 := sq_nonneg s
  have hx1 : s ^ 2 < 1 := by nlinarith
  have ha : 0 < 1 - s ^ 2 := by linarith
  have hg := hasSum_geometric_of_lt_one hx0 hx1
  have hn := hasSum_coe_mul_geometric_of_norm_lt_one
    (show ‖s ^ 2‖ < 1 by rw [Real.norm_of_nonneg hx0]; exact hx1)
  have hsum := (hn.add hg).mul_left ((1 - s ^ 2) * (t - s) ^ 2)
  unfold Ft
  have hterm : ∀ n : ℕ, ((n : ℝ) + 1) * (t * zs s n - zs s (n + 1)) ^ 2
      = (1 - s ^ 2) * (t - s) ^ 2 * ((n : ℝ) * (s ^ 2) ^ n + (s ^ 2) ^ n) := by
    intro n
    have e : t * zs s n - zs s (n + 1) = (t - s) * zs s n := by unfold zs; ring
    rw [e, mul_pow, zs_sq hs1 hs0 n]; ring
  simp only [hterm]
  rw [hsum.tsum_eq]
  field_simp
  ring

/-! ## The supporting-line inequality (S:support) -/

/-- `φ(s) = 𝓔(s) + λ (t−s)²/(1−s²)`. -/
def phi (L t s : ℝ) : ℝ := EE s + L * ((t - s) ^ 2 / (1 - s ^ 2))

/-- A continuous closed form of `φ` on `[0,1)`. -/
def psi (L t s : ℝ) : ℝ :=
  -log (1 - s ^ 2) - 2 * s * (s * log s) / (1 - s ^ 2) + L * ((t - s) ^ 2 / (1 - s ^ 2))

lemma phi_eq_psi (L t : ℝ) {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) : phi L t s = psi L t s := by
  unfold phi psi
```

```
  rcases hs0.eq_or_lt with h0 | hpos
  · subst h0; rw [EE_zero]; simp
  · rw [EE_formula hpos hs1]; ring

lemma psi_continuousAt (L t : ℝ) {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) :
    ContinuousAt (psi L t) s := by
  have hd : 1 - s ^ 2 ≠ 0 := by nlinarith
  unfold psi
  have c1 : ContinuousAt (fun s : ℝ => 1 - s ^ 2) s := by fun_prop
  have c2 : ContinuousAt (fun s : ℝ => s * log s) s := continuous_mul_log.continuousAt
  refine ((ContinuousAt.neg (c1.log hd)).sub ?_).add (continuousAt_const.mul ?_)
  · exact ((continuousAt_const.mul continuousAt_id).mul c2).div c1 hd
  · exact (((continuousAt_const.sub continuousAt_id).pow 2)).div c1 hd

/-- `g(s) = −2 s ln s − λ (t−s)(1−ts)`; `φ'(s) = 2 g(s)/(1−s²)²`. -/
def gsign (L t s : ℝ) : ℝ := -2 * s * log s - L * (t - s) * (1 - t * s)

lemma psi_hasDerivAt (L t : ℝ) {s : ℝ} (hs0 : 0 < s) (hs1 : s < 1) :
    HasDerivAt (psi L t) (2 * gsign L t s / (1 - s ^ 2) ^ 2) s := by
  have hd : 1 - s ^ 2 ≠ 0 := by nlinarith
  have d1 : HasDerivAt (fun s : ℝ => 1 - s ^ 2) (-(2 * s)) s := by
    simpa using (hasDerivAt_pow 2 s).const_sub 1
  have d2 : HasDerivAt (fun s : ℝ => s * log s) (log s + 1) s := hasDerivAt_mul_log hs0.ne'
  have d3 := ((hasDerivAt_id' s).const_mul (2:ℝ)).mul d2
  have d4 := ((hasDerivAt_id' s).const_sub t).pow 2
  have h := ((d1.log hd).neg.sub (d3.div d1 hd)).add ((d4.div d1 hd).const_mul L)
  unfold psi
  convert h using 1
  unfold gsign
  simp only [Pi.mul_apply, Pi.pow_apply]
  field_simp
  ring

/-- `k(s) = −s ln s/(1−s)` is strictly increasing on `(0,1)`. -/
lemma k_strictMono : StrictMonoOn (fun s : ℝ => -(s * log s) / (1 - s)) (Set.Ioo 0 1) := by
  apply strictMonoOn_of_deriv_pos (convex_Ioo 0 1)
  · intro s hs
    have : 1 - s ≠ 0 := by linarith [hs.2]
    exact (continuous_mul_log.continuousAt.neg.div (continuousAt_const.sub continuousAt_id)
      this).continuousWithinAt
  · intro s hs
    rw [interior_Ioo] at hs
    have h1 : 1 - s ≠ 0 := by linarith [hs.2]
    have hd := ((hasDerivAt_mul_log hs.1.ne').neg).div ((hasDerivAt_id' s).const_sub 1) h1
    rw [show deriv (fun s : ℝ => -(s * log s) / (1 - s)) s
        = (-(log s + 1) * (1 - s) - (-(s * log s)) * -1) / (1 - s) ^ 2 from hd.deriv]
    have hl := Real.log_lt_sub_one_of_pos hs.1 (by linarith [hs.2] : s ≠ 1)
    have : 0 < (1 - s) ^ 2 := by positivity
    apply div_pos _ this
    nlinarith

/-- `Λ_t(s) = −2 s ln s/((t−s)(1−ts))` is strictly increasing on `(0,t)` (Eq. S:LambdaMon). -/
lemma Lambda_strictMono {t : ℝ} (ht0 : 0 < t) (ht1 : t ≤ 1) :
    StrictMonoOn (fun s : ℝ => -2 * s * log s / ((t - s) * (1 - t * s))) (Set.Ioo 0 t) := by
  intro s1 hs1 s2 hs2 h12
  have hs1' : s1 ∈ Set.Ioo (0:ℝ) 1 := (hs1.1, by linarith [hs1.2])
  have hs2' : s2 ∈ Set.Ioo (0:ℝ) 1 := (hs2.1, by linarith [hs2.2])
  have hk := k_strictMono hs1' hs2' h12
  simp only at hk
  have hk1 : 0 < -(s1 * log s1) / (1 - s1) := by
    apply div_pos _ (by linarith [hs1'.2])
    have := Real.log_neg hs1'.1 hs1'.2
    nlinarith [hs1'.1]
  -- m(s) = (1−s)/((t−s)(1−ts)) is positive and nondecreasing
  have hm1 : 0 < (1 - s1) / ((t - s1) * (1 - t * s1)) := by
    apply div_pos (by linarith [hs1'.2]) (mul_pos (by linarith [hs1.2]) (by nlinarith [hs1'.2]))
  have hm : (1 - s1) / ((t - s1) * (1 - t * s1)) ≤ (1 - s2) / ((t - s2) * (1 - t * s2)) := by
    have a1 : 0 < t - s1 := by linarith [hs1.2]
    have a2 : 0 < t - s2 := by linarith [hs2.2]
    have b1 : 0 < 1 - t * s1 := by nlinarith [hs1'.2]
    have b2 : 0 < 1 - t * s2 := by nlinarith [hs2'.2]
    rw [div_le_div_iff₀ (mul_pos a1 b1) (mul_pos a2 b2)]
    have e1 : (1 - s1) * (t - s2) ≤ (1 - s2) * (t - s1) := by nlinarith
    have e2 : 1 - t * s2 ≤ 1 - t * s1 := by nlinarith
    have := mul_le_mul e1 e2 b2.le (by nlinarith [hs2'.2])
    nlinarith
  have f1 : -2 * s1 * log s1 / ((t - s1) * (1 - t * s1))
      = 2 * (-(s1 * log s1) / (1 - s1)) * ((1 - s1) / ((t - s1) * (1 - t * s1))) := by
    have : 1 - s1 ≠ 0 := by linarith [hs1'.2]
    field_simp
  have f2 : -2 * s2 * log s2 / ((t - s2) * (1 - t * s2))
      = 2 * (-(s2 * log s2) / (1 - s2)) * ((1 - s2) / ((t - s2) * (1 - t * s2))) := by
    have : 1 - s2 ≠ 0 := by linarith [hs2'.2]
    field_simp
  simp only
  rw [f1, f2]
  have := mul_lt_mul hk hm hm1 (by linarith)
  nlinarith

/-- Sign of `g`: negative on `(0,q)`, positive on `(q,1)`. -/
lemma gsign_cases {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) {s : ℝ} (hs0 : 0 < s)
```

```
    (hs1 : s < 1) :
    (s < q → gsign (lam q t) t s < 0) ∧ (q < s → 0 < gsign (lam q t) t s) := by
  have hL := lam_pos hq hqt ht
  have hΛq : -2 * q * log q / ((t - q) * (1 - t * q)) = lam q t := by
    unfold lam; ring_nf
  have hfac : ∀ s, 0 < s → s < t →
      gsign (lam q t) t s = (t - s) * (1 - t * s)
        * (-2 * s * log s / ((t - s) * (1 - t * s)) - lam q t) := by
    intro s h0 h1
    have e1 : t - s ≠ 0 := by linarith
    have e2 : 1 - t * s ≠ 0 := by nlinarith
    have e3 : 1 - s * t ≠ 0 := by rw [mul_comm]; exact e2
    unfold gsign; field_simp
  have ht0 : 0 < t := hq.trans hqt
  constructor
  · intro hsq
    have hst : s < t := hsq.trans hqt
    rw [hfac s hs0 hst]
    have := Lambda_strictMono ht0 ht ⟨hs0, hst⟩ ⟨hq, hqt⟩ hsq
    simp only at this
    rw [hΛq] at this
    exact mul_neg_of_pos_of_neg (mul_pos (by linarith) (by nlinarith)) (by linarith)
  · intro hqs
    rcases lt_or_ge s t with hst | hst
    · rw [hfac s hs0 hst]
      have := Lambda_strictMono ht0 ht ⟨hq, hqt⟩ ⟨hs0, hst⟩ hqs
      simp only at this
      rw [hΛq] at this
      exact mul_pos (mul_pos (by linarith) (by nlinarith)) (by linarith)
    · unfold gsign
      have hl : log s < 0 := Real.log_neg hs0 hs1
      have : 0 ≤ (s - t) * (1 - t * s) := mul_nonneg (by linarith) (by nlinarith)
      nlinarith [mul_pos hs0 (neg_pos.mpr hl)]

/-- **Eq. (S:support)**: for `0 < q < t ≤ 1` and `λ = λ_{q,t}`,
`𝓔(s) + λ(t−s)²/(1−s²) ≥ 𝓔(q) + λ(t−q)²/(1−q²)` for all `0 ≤ s < 1`. -/
theorem support {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) {s : ℝ} (hs0 : 0 ≤ s)
    (hs1 : s < 1) : phi (lam q t) t q ≤ phi (lam q t) t s := by
  have hq1 : q < 1 := lt_of_lt_of_le hqt ht
  rw [phi_eq_psi _ _ hq.le hq1, phi_eq_psi _ _ hs0 hs1]
  have hderiv : ∀ x, 0 < x → x < 1 →
      deriv (psi (lam q t) t) x = 2 * gsign (lam q t) t x / (1 - x ^ 2) ^ 2 :=
    fun x h0 h1 => (psi_hasDerivAt _ _ h0 h1).deriv
  have hdiff : ∀ x, 0 < x → x < 1 → DifferentiableAt ℝ (psi (lam q t) t) x :=
    fun x h0 h1 => (psi_hasDerivAt _ _ h0 h1).differentiableAt
  rcases le_total s q with hsq | hsq
  · -- antitone on [0, q]
    have hanti : AntitoneOn (psi (lam q t) t) (Set.Icc 0 q) := by
      apply antitoneOn_of_deriv_nonpos (convex_Icc 0 q)
      · intro x hx
        exact (psi_continuousAt _ _ hx.1 (lt_of_le_of_lt hx.2 hq1)).continuousWithinAt
      · intro x hx
        rw [interior_Icc] at hx
        exact (hdiff x hx.1 (hx.2.trans hq1)).differentiableWithinAt
      · intro x hx
        rw [interior_Icc] at hx
        rw [hderiv x hx.1 (hx.2.trans hq1)]
        have := (gsign_cases hq hqt ht hx.1 (hx.2.trans hq1)).1 hx.2
        have : 0 < (1 - x ^ 2) ^ 2 := by
          have : 0 < 1 - x ^ 2 := by nlinarith [hx.2.trans hq1, hx.1]
          positivity
        apply div_nonpos_of_nonpos_of_nonneg (by linarith) this.le
    exact hanti ⟨hs0, hsq⟩ ⟨hq.le, le_rfl⟩ hsq
  · -- monotone on [q, 1)
    have hmono : MonotoneOn (psi (lam q t) t) (Set.Ico q 1) := by
      apply monotoneOn_of_deriv_nonneg (convex_Ico q 1)
      · intro x hx
        exact (psi_continuousAt _ _ (hq.le.trans hx.1) hx.2).continuousWithinAt
      · intro x hx
        rw [interior_Ico] at hx
        exact (hdiff x (hq.trans hx.1) hx.2).differentiableWithinAt
      · intro x hx
        rw [interior_Ico] at hx
        rw [hderiv x (hq.trans hx.1) hx.2]
        have := (gsign_cases hq hqt ht (hq.trans hx.1) hx.2).2 hx.1
        apply div_nonneg (by linarith) (by positivity)
    exact hmono ⟨le_rfl, hq1⟩ ⟨hsq, hs1⟩ hsq

end Thm1Cert
```

Thm1Cert/Assembly.lean

```
/-
Assembly of Theorem 1 of G. Adesso, "Optimality of Gaussian entanglement of formation"
(arXiv:2608.01909v3), Supplemental Material Sec. "Completion of the proof":

  * `thm1_spectrum`: for every decreasing Schmidt spectrum c with ∑ c_n² = 1 and finite entropy,
        𝓔(q) + λ_{q,t}(t−q)²/(1−q²) ≤ H(c²) + λ_{q,t} ∑ (n+1) d_n²
    (compression on normalized truncations, the closure, and the limit M → ∞);
```

```
  * `theorem1`: with the defect inequality (Theorem S1),
        E(ψ) + λ_{q,t} [D_t(ψ) − d_t(q)] ≥ 𝓔(q),     D_t = G_t + 1 − t²   (Eq. S:Gdef),
    in the Schmidt-basis matrix model of `Defect.lean`; the identifications E(ψ) = H(c²)
    (Eq. S:entropy), D_t = G_t + 1 − t² and the matrix model itself are inputs;
  * `theorem1_endpoint`: the case q = t < 1 (Eq. S:endpoint);
  * `fixed_entanglement`: E(ψ) ≤ 𝓔(q) ⇒ D_t(ψ) ≥ d_t(q) (Eq. S:fixedE).

Input: `BRK t λ` − the Beigi–Rahimi-Keshari meta log-Sobolev theorem, restricted to diagonal
states of finite rank: H(z²) + λ 𝓕_t(z) ≥ inf_{0≤s<1} [H((z^{(s)})²) + λ 𝓕_t(z^{(s)})].
-/
import Mathlib.Algebra.BigOperators.Field
import Thm1Cert.Closure

open Finset Real Filter Topology

set_option autoImplicit false

noncomputable section

namespace Thm1Cert

open Thm2Cert (EE gfun lam dt EE_formula lam_pos)

/-- **Input (Beigi–Rahimi-Keshari, diagonal states of finite rank).** For every finitely
supported `z ≥ 0` with `∑ z_n² = 1`, `H(z²) + λ 𝓕_t(z)` is at least the infimum over thermal
states. (Y(ρ_z) = H(z²) + λ𝓕_t(z) + const for the parameters (S:parameters).) -/
def BRK (t L : ℝ) : Prop :=
  ∀ z : ℕ → ℝ, (∀ n, 0 ≤ z n) → (∃ M, ∀ n, M < n → z n = 0) → HasSum (fun n => z n ^ 2) 1 →
    ∀ c0 : ℝ, (∀ s, 0 ≤ s → s < 1 → c0 ≤ Hs (zs s) + L * Ft t (zs s)) → c0 ≤ Hs z + L * Ft t z

variable {q t : ℝ}

/-- Closure for a compressed, finitely supported sequence. -/
lemma closure_finite (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (M : ℕ) (z : ℕ → ℝ) (hz0 : ∀ n, 0 ≤ z n) (hzs : ∀ n, M < n → z n = 0)
    (hzn : ∑ n ∈ range (M + 1), z n ^ 2 = 1) :
    phi (lam q t) t q ≤ ∑ n ∈ range (M + 1), negMulLog (z n ^ 2)
      + lam q t * ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2 := by
  have hout : ∀ n, n ∉ range (M + 1) → M < n := fun n hn => by simp at hn; omega
  have hH : Hs z = ∑ n ∈ range (M + 1), negMulLog (z n ^ 2) := by
    unfold Hs
    exact tsum_eq_sum (fun n hn => by rw [hzs n (hout n hn)]; simp)
  have hF : Ft t z = ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2 := by
    unfold Ft
    exact tsum_eq_sum (fun n hn => by
      rw [hzs n (hout n hn), hzs (n + 1) (by have := hout n hn; omega)]; ring)
  have hsum : HasSum (fun n => z n ^ 2) 1 := by
    rw [← hzn]
    exact hasSum_sum_of_ne_finset_zero (fun n hn => by rw [hzs n (hout n hn)]; ring)
  have := hB z hz0 ⟨M, hzs⟩ hsum (phi (lam q t) t q) (fun s hs0 hs1 => by
    rw [H_thermal hs0 hs1, F_thermal t hs0 hs1]
    exact support hq hqt ht hs0 hs1)
  rwa [hH, hF] at this

lemma negMulLog_div {x P : ℝ} (hx : 0 < x) (hP : 0 < P) :
    negMulLog (x / P) = negMulLog x / P + x / P * log P := by
  unfold negMulLog
  rw [Real.log_div hx.ne' hP.ne']
  field_simp
  ring

/-- **Truncation bound.** For `c_M > 0` and `P = ∑_{n ≤ M} c_n²`, the normalized truncation gives
`φ(q) ≤ S_M/P + ln P + λ (∑_{n<M}(n+1)d_n² + (M+1)t² c_M²)/P`. -/
lemma trunc_bound (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (_hc0 : ∀ n, 0 ≤ c n) (M : ℕ) (hcM : 0 < c M) :
    phi (lam q t) t q
      ≤ (∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) / (∑ n ∈ range (M + 1), c n ^ 2)
        + log (∑ n ∈ range (M + 1), c n ^ 2)
        + lam q t * ((∑ n ∈ range M, ((n : ℝ) + 1) * dd t c n ^ 2
            + ((M : ℝ) + 1) * (t * c M) ^ 2) / (∑ n ∈ range (M + 1), c n ^ 2)) := by
  classical
  have ht0 : 0 < t := hq.trans hqt
  have hpos : ∀ n, n ≤ M → 0 < c n := fun n hn => lt_of_lt_of_le hcM (hc hn)
  set P := ∑ n ∈ range (M + 1), c n ^ 2 with hP
  have hP0 : 0 < P := sum_pos (fun n hn => pow_pos (hpos n (by simp at hn; omega)) 2)
    ⟨0, by simp⟩
  have hsP : 0 < √P := Real.sqrt_pos.mpr hP0
  set cM : ℕ → ℝ := fun n => if n ≤ M then c n / √P else 0 with hcM_def
  have hcMpos : ∀ n, n ≤ M → 0 < cM n := fun n hn => by
    simp only [hcM_def, ite_t hn]; exact div_pos (hpos n hn) hsP
  have hcManti : ∀ n, n < M → cM (n + 1) ≤ cM n := fun n hn => by
    simp only [hcM_def, ite_t (by omega : n + 1 ≤ M), ite_t hn.le]
    exact div_le_div_of_nonneg_right (hc (Nat.le_succ n)) hsP.le
  have hcMsupp : ∀ n, M < n → cM n = 0 := fun n hn => by
    simp only [hcM_def, show ¬ n ≤ M by omega, ↓reduceIte]
  have hsq : ∀ n, n ≤ M → cM n ^ 2 = c n ^ 2 / P := fun n hn => by
    simp only [hcM_def, ite_t hn]; rw [div_pow, Real.sq_sqrt hP0.le]
  have hcMnorm : ∑ n ∈ range (M + 1), cM n ^ 2 = 1 := by
    rw [sum_congr rfl (fun n hn => hsq n (by simp at hn; omega)), ← sum_div, div_self hP0.ne']
  obtain ⟨z, hz0, hzs, hzn, -, hzH, hzE⟩ :=
    compression ht0 cM hcMpos hcManti hcMsupp hcMnorm
```

```
  have hcl := closure_finite hq hqt ht hB M z hz0 hzs hzn
  have hL := (lam_pos hq hqt ht).le
  -- entropy of the truncation
  have hent : ∑ n ∈ range (M + 1), negMulLog (cM n ^ 2)
      = (∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) / P + log P := by
    rw [sum_congr rfl (fun n hn => by
      rw [hsq n (by simp at hn; omega), negMulLog_div (pow_pos (hpos n (by simp at hn; omega)) 2) hP0])]
    rw [sum_add_distrib, ← sum_div, ← sum_mul, ← sum_div, ← hP, div_self hP0.ne', one_mul]
  -- energy of the truncation
  have hen : ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * pos (t * cM n - cM (n + 1)) ^ 2
      = (∑ n ∈ range M, ((n : ℝ) + 1) * dd t c n ^ 2 + ((M : ℝ) + 1) * (t * c M) ^ 2) / P := by
    rw [sum_range_succ, add_div, sum_div]
    congr 1
    · refine sum_congr rfl (fun n hn => ?_)
      simp only [mem_range] at hn
      simp only [hcM_def, ite_t hn.le, ite_t (by omega : n + 1 ≤ M)]
      rw [show t * (c n / √P) - c (n + 1) / √P = (1 / √P) * (t * c n - c (n + 1)) by
        field_simp]
      rw [pos_mul_left (by positivity), mul_pow, div_pow, one_pow, Real.sq_sqrt hP0.le]
      unfold dd; field_simp
    · simp only [hcM_def, ite_t le_rfl, show ¬ M + 1 ≤ M by omega, ↓reduceIte, sub_zero]
      have h0 : 0 ≤ t * (c M / √P) := by positivity
      rw [show pos (t * (c M / √P)) = t * (c M / √P) from max_eq_left h0, mul_pow, div_pow,
        Real.sq_sqrt hP0.le]
      field_simp
  calc phi (lam q t) t q ≤ _ := hcl
    _ ≤ ∑ n ∈ range (M + 1), negMulLog (cM n ^ 2)
        + lam q t * ∑ n ∈ range (M + 1), ((n : ℝ) + 1) * pos (t * cM n - cM (n + 1)) ^ 2 :=
      add_le_add hzH (mul_le_mul_of_nonneg_left hzE hL)
    _ = _ := by rw [hent, hen]

/-- For decreasing summable `p ≥ 0`, `(M+1) p_M → 0`. -/
lemma tendsto_mul_antitone (p : ℕ → ℝ) (hp0 : ∀ n, 0 ≤ p n) (hanti : Antitone p) {A : ℝ}
    (hs : HasSum p A) : Tendsto (fun M : ℕ => ((M : ℝ) + 1) * p M) atTop (𝓝 0) := by
  set tail : ℕ → ℝ := fun k => A - ∑ n ∈ range k, p n with htail
  have htl : Tendsto tail atTop (𝓝 0) := by
    have := (tendsto_const_nhds (x := A)).sub hs.tendsto_sum_nat
    simpa [htail] using this
  have hdiv : Tendsto (fun M : ℕ => M / 2) atTop atTop :=
    tendsto_atTop_atTop.mpr (fun b => (2 * b, fun a ha => by omega))
  have hup : Tendsto (fun M : ℕ => 2 * tail (M / 2)) atTop (𝓝 0) := by
    simpa using (htl.comp hdiv).const_mul 2
  refine tendsto_of_tendsto_of_tendsto_of_le_of_le tendsto_const_nhds hup
    (fun M => mul_nonneg (by positivity) (hp0 M)) (fun M => ?_)
  simp only
  set k := M / 2
  have hk : k ≤ M + 1 := by omega
  have h1 : ((M : ℝ) + 1 - k) * p M ≤ ∑ n ∈ Ico k (M + 1), p n := by
    have : ∑ n ∈ Ico k (M + 1), p M ≤ ∑ n ∈ Ico k (M + 1), p n :=
      sum_le_sum (fun n hn => hanti (by simp at hn; omega))
    rw [sum_const, Nat.card_Ico, nsmul_eq_mul] at this
    rw [Nat.cast_sub hk] at this
    push_cast at this
    linarith
  have h2 : ∑ n ∈ range (M + 1), p n ≤ A := sum_le_hasSum _ (fun n _ => hp0 n) hs
  have h3 := sum_range_add_sum_Ico p hk
  have h4 : ((M : ℝ) + 1) ≤ 2 * ((M : ℝ) + 1 - k) := by
    have : 2 * k ≤ M := Nat.mul_div_le M 2
    have : (2 : ℝ) * k ≤ M := by exact_mod_cast this
    linarith
  have h5 := mul_le_mul_of_nonneg_right h4 (hp0 M)
  simp only [htail]
  nlinarith

/-- **Theorem 1, spectral form.** -/
theorem thm1_spectrum (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (hD : Summable (fun n : ℕ => ((n : ℝ) + 1) * dd t c n ^ 2)) :
    phi (lam q t) t q
      ≤ ∑' n, negMulLog (c n ^ 2) + lam q t * ∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2 := by
  classical
  have hL := (lam_pos hq hqt ht).le
  by_cases hfin : ∃ R, c (R + 1) = 0
  · -- finite Schmidt rank
    set R := Nat.find hfin with hR
    have hR1 : c (R + 1) = 0 := Nat.find_spec hfin
    have hzero : ∀ n, R < n → c n = 0 := fun n hn =>
      le_antisymm (hR1 ▸ hc (by omega)) (hc0 n)
    have hcR : 0 < c R := by
      rcases (hc0 R).eq_or_lt with h | h
      · exfalso
        rcases Nat.eq_zero_or_pos R with h0 | h0
        · -- c 0 = 0 forces c ≡ 0, contradicting normalization
          have hall : ∀ n, c n = 0 := fun n =>
            le_antisymm ((hc (Nat.zero_le n)).trans (by rw [← h0]; exact h.symm.le)) (hc0 n)
          have : HasSum (fun n => c n ^ 2) 0 := by simp [hall]
          exact one_ne_zero (hnorm.unique this)
        · exact Nat.find_min hfin (show R - 1 < R by omega)
            (by rw [show R - 1 + 1 = R by omega]; exact h.symm)
      · exact h
```

```
    have hb := trunc_bound hq hqt ht hB c hc hc0 R hcR
    have hout : ∀ n, n ∉ range (R + 1) → R < n := fun n hn => by simp at hn; omega
    have hP : ∑ n ∈ range (R + 1), c n ^ 2 = 1 := by
      rw [← hnorm.tsum_eq]
      exact (tsum_eq_sum (fun n hn => by rw [hzero n (hout n hn)]; ring)).symm
    have hS : ∑' n, negMulLog (c n ^ 2) = ∑ n ∈ range (R + 1), negMulLog (c n ^ 2) :=
      tsum_eq_sum (fun n hn => by rw [hzero n (hout n hn)]; simp)
    have hDs : ∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2
        = ∑ n ∈ range R, ((n : ℝ) + 1) * dd t c n ^ 2 + ((R : ℝ) + 1) * (t * c R) ^ 2 := by
      rw [tsum_eq_sum (s := range (R + 1)) (fun n hn => by
        unfold dd; rw [hzero n (hout n hn), hzero (n + 1) (by have := hout n hn; omega)]
        simp [pos]), sum_range_succ]
      unfold dd; rw [hR1, sub_zero, show pos (t * c R) = t * c R from
        max_eq_left (mul_nonneg (hq.trans hqt).le (hc0 R))]
    rw [hP, Real.log_one, div_one, div_one, add_zero] at hb
    rw [hS, hDs]; exact hb
  · -- infinite Schmidt rank: pass to the limit M → ∞
    simp only [not_exists] at hfin
    have hcpos : ∀ M, 0 < c M := fun M =>
      lt_of_lt_of_le ((hc0 (M + 1)).lt_of_ne (Ne.symm (hfin M))) (hc (Nat.le_succ M))
    have hP : Tendsto (fun M : ℕ => ∑ n ∈ range (M + 1), c n ^ 2) atTop (𝓝 1) :=
      (hnorm.tendsto_sum_nat).comp (tendsto_add_atTop_nat 1)
    have hS : Tendsto (fun M : ℕ => ∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) atTop
        (𝓝 (∑' n, negMulLog (c n ^ 2))) :=
      (hH.hasSum.tendsto_sum_nat).comp (tendsto_add_atTop_nat 1)
    have hDt : Tendsto (fun M : ℕ => ∑ n ∈ range M, ((n : ℝ) + 1) * dd t c n ^ 2) atTop
        (𝓝 (∑' n : ℕ, ((n : ℝ) + 1) * dd t c n ^ 2)) := hD.hasSum.tendsto_sum_nat
    have hsqanti : Antitone (fun n => c n ^ 2) := fun a b hab =>
      pow_le_pow_left₀ (hc0 b) (hc hab) 2
    have hB0 : Tendsto (fun M : ℕ => ((M : ℝ) + 1) * (t * c M) ^ 2) atTop (𝓝 0) := by
      have := (tendsto_mul_antitone (fun n => c n ^ 2) (fun n => sq_nonneg _) hsqanti hnorm).const_mul
        (t ^ 2)
      simp only [mul_zero] at this
      refine this.congr (fun M => ?_)
      ring
    have hlog : Tendsto (fun M : ℕ => log (∑ n ∈ range (M + 1), c n ^ 2)) atTop (𝓝 0) := by
      rw [← Real.log_one]
      exact (Real.continuousAt_log one_ne_zero).tendsto.comp hP
    have hlim := ((hS.div hP one_ne_zero).add hlog).add
      (((hDt.add hB0).div hP one_ne_zero).const_mul (lam q t))
    simp only [div_one, add_zero] at hlim
    exact ge_of_tendsto hlim (Eventually.of_forall
      (fun M => trunc_bound hq hqt ht hB c hc hc0 M (hcpos M)))

lemma dt_identity (hq0 : 0 ≤ q) (hq1 : q < 1) : dt t q - 1 + t ^ 2 = (t - q) ^ 2 / (1 - q ^ 2) := by
  have : 1 - q ^ 2 ≠ 0 := by nlinarith [sq_nonneg q]
  unfold dt; field_simp; ring

/-- **Theorem 1** (affine extremality), matrix model. With `E(ψ) = H(c²)` (Eq. S:entropy) and
`D_t(ψ) = G + 1 − t²` (Eq. S:Gdef), where `G = ∑ |t c_i X_ij − c_j Y_ij|²` (Eq. S:Gmatrix):
`E(ψ) + λ_{q,t} [D_t(ψ) − d_t(q)] ≥ 𝓔(q)` for all `0 < q < t ≤ 1`. -/
theorem theorem1 (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) (hB : BRK t (lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    EE q ≤ ∑' n, negMulLog (c n ^ 2) + lam q t * ((G + 1 - t ^ 2) - dt t q) := by
  have ht0 : 0 < t := hq.trans hqt
  obtain ⟨hDs, hDle⟩ := defect_inequality_tsum ht0 ht hc hc0 X Y G hG rX cX rY cY
    hrX hcX hrY hcY hX hY
  have h := thm1_spectrum hq hqt ht hB c hc hc0 hnorm hH hDs
  have hL := (lam_pos hq hqt ht).le
  have hq1 : q < 1 := lt_of_lt_of_le hqt ht
  unfold phi at h
  rw [show (G + 1 - t ^ 2) - dt t q = G - (dt t q - 1 + t ^ 2) by ring, dt_identity hq.le hq1]
  have := mul_le_mul_of_nonneg_left hDle hL
  nlinarith

/-- **Eq. (S:fixedE)**: `E(ψ) ≤ 𝓔(q)` implies `D_t(ψ) ≥ d_t(q)`. -/
theorem fixed_entanglement (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1) {E D : ℝ}
    (h1 : EE q ≤ E + lam q t * (D - dt t q)) (hE : E ≤ EE q) : dt t q ≤ D := by
  have hL := lam_pos hq hqt ht
  by_contra hc
  rw [not_le] at hc
  have : lam q t * (D - dt t q) < 0 := mul_neg_of_pos_of_neg hL (by linarith)
  linarith

lemma EE_continuousAt {s : ℝ} (hs0 : 0 < s) (hs1 : s < 1) : ContinuousAt EE s := by
  have hd : 1 - s ^ 2 ≠ 0 := by nlinarith
  have heq : (fun x => -log (1 - x ^ 2) - 2 * (x ^ 2 / (1 - x ^ 2)) * log x) =ᶠ[𝓝 s] EE := by
    have : Set.Ioo (0:ℝ) 1 ∈ 𝓝 s := Ioo_mem_nhds hs0 hs1
    filter_upwards [this] with x hx
    rw [EE_formula hx.1 hx.2]
  refine ContinuousAt.congr ?_ heq
```

```
  have c1 : ContinuousAt (fun x : ℝ => 1 - x ^ 2) s := by fun_prop
  exact (c1.log hd).neg.sub ((continuousAt_const.mul
    ((continuousAt_id.pow 2).div c1 hd)).mul (Real.continuousAt_log hs0.ne'))

/-- **Eq. (S:endpoint)**: for `t < 1`, if the affine bound holds for all `q < t` and
`D_t(ψ) = d_t(t) = 1 − t²`, then `E(ψ) ≥ 𝓔(t)`. -/
theorem theorem1_endpoint (ht0 : 0 < t) (ht1 : t < 1) {E : ℝ}
    (h : ∀ q, 0 < q → q < t → EE q ≤ E + lam q t * (dt t t - dt t q)) : EE t ≤ E := by
  have hle : ∀ q, 0 < q → q < t → EE q ≤ E := by
    intro q hq hqt
    have hL := (lam_pos hq hqt ht1.le).le
    have hq1 : q < 1 := hqt.trans ht1
    have e : dt t t - dt t q = -((t - q) ^ 2 / (1 - q ^ 2)) := by
      have a1 : 1 - q ^ 2 ≠ 0 := by nlinarith
      have a2 : 1 - t ^ 2 ≠ 0 := by nlinarith
      unfold dt; field_simp; ring
    have : 0 ≤ (t - q) ^ 2 / (1 - q ^ 2) := div_nonneg (sq_nonneg _) (by nlinarith)
    have := h q hq hqt
    rw [e] at this
    nlinarith
  have hc := (EE_continuousAt ht0 ht1).tendsto.mono_left (nhdsWithin_le_nhds (s := Set.Ioo 0 t))
  have hev : ∀ᶠ q in 𝓝[Set.Ioo 0 t] t, EE q ≤ E :=
    eventually_nhdsWithin_of_forall (fun q hq => hle q hq.1 hq.2)
  have : (𝓝[Set.Ioo 0 t] t).NeBot := by
    rw [← mem_closure_iff_nhdsWithin_neBot, closure_Ioo ht0.ne]
    exact ⟨ht0.le, le_rfl⟩
  exact le_of_tendsto hc hev

end Thm1Cert
```

Thm1Cert/Upsilon.lean

```
/-
The Beigi–Rahimi-Keshari input in the form used in the Supplemental Material (Eqs. S:Upsilon,
S:parameters, S:Fmoments, S:UpsilonF), for diagonal states ρ_z = ∑ z_n² |n⟩⟨n|:

    Y(ρ_z) = 2ν₀[(ñ+1) − T_z] + ω ñ + H(z²),   ν₀ = λt, ν₁ = 0, ω = λ(1−t)²,
    ñ = ∑ n z_n²,   T_z = ∑ (n+1) z_n z_{n+1}.

Certified: 𝓕_t(z) = t²(ñ+1) + ñ − 2t T_z (S:Fmoments) and Y(ρ_z) = H(z²) + λ𝓕_t(z) + λ(2t − t²)
(S:UpsilonF), both for finitely supported z and for the thermal sequences z^{(s)}; hence the
statement "inf Y is attained on thermal states" (`BRKUpsilon`, restricted to diagonal states of
finite rank) implies the sequence form `BRK` used in `Assembly.lean`.
The diagonal-state moment formulas Tr(ρ_z a†a) = ñ, Tr(√ρ_z a √ρ_z a†) = T_z are inputs.
-/
import Thm1Cert.Assembly

open Finset Real Filter Topology

set_option autoImplicit false

noncomputable section

namespace Thm1Cert

/-- Mean photon number `ñ = ∑ n z_n²`. -/
def nbar (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, (n : ℝ) * z n ^ 2

/-- `T_z = ∑ (n+1) z_n z_{n+1} = Tr(√ρ_z a √ρ_z a†)` (Eq. S:Tz). -/
def Tz (z : ℕ → ℝ) : ℝ := ∑' n : ℕ, ((n : ℝ) + 1) * z n * z (n + 1)

/-- `Y(ρ_z)` for `ν₀ = λt`, `ν₁ = 0`, `ω = λ(1−t)²` (Eqs. S:Upsilon, S:parameters). -/
def Ups (t L : ℝ) (z : ℕ → ℝ) : ℝ :=
  2 * (L * t) * ((nbar z + 1) - Tz z) + L * (1 - t) ^ 2 * nbar z + Hs z

/-- **Input (Beigi–Rahimi-Keshari)**, restricted to diagonal states of finite rank:
`Y(ρ_z) ≥ inf_{0 ≤ s < 1} Y(τ_s)`. -/
def BRKUpsilon (t L : ℝ) : Prop :=
  ∀ z : ℕ → ℝ, (∀ n, 0 ≤ z n) → (∃ M, ∀ n, M < n → z n = 0) → HasSum (fun n => z n ^ 2) 1 →
    ∀ c0 : ℝ, (∀ s, 0 ≤ s → s < 1 → c0 ≤ Ups t L (zs s)) → c0 ≤ Ups t L z

/-- **Eq. (S:Fmoments)** in `HasSum` form. -/
lemma Fmoments (t : ℝ) (z : ℕ → ℝ) {N T : ℝ} (h1 : HasSum (fun n => z n ^ 2) 1)
    (hN : HasSum (fun n : ℕ => (n : ℝ) * z n ^ 2) N)
    (hT : HasSum (fun n : ℕ => ((n : ℝ) + 1) * z n * z (n + 1)) T) :
    HasSum (fun n : ℕ => ((n : ℝ) + 1) * (t * z n - z (n + 1)) ^ 2)
      (t ^ 2 * (N + 1) + N - 2 * t * T) := by
  have hA : HasSum (fun n : ℕ => ((n : ℝ) + 1) * z n ^ 2) (N + 1) := by
    have := hN.add h1; simpa [add_mul] using this
  have hB : HasSum (fun n : ℕ => ((n : ℝ) + 1) * z (n + 1) ^ 2) N := by
    have := (hasSum_nat_add_iff' 1).mpr hN
    simp only [range_one, sum_singleton, Nat.cast_zero, zero_mul, sub_zero] at this
    simpa using this
  have := ((hA.mul_left (t ^ 2)).add hB).sub (hT.mul_left (2 * t))
  convert this using 1
  funext n; ring

/-- **Eq. (S:UpsilonF)**: `Y(ρ_z) = H(z²) + λ𝓕_t(z) + λ(2t − t²)`. -/
lemma Ups_eq (t L : ℝ) (z : ℕ → ℝ) (h1 : HasSum (fun n => z n ^ 2) 1)
```

```
    (hN : Summable (fun n : ℕ => (n : ℝ) * z n ^ 2))
    (hT : Summable (fun n : ℕ => ((n : ℝ) + 1) * z n * z (n + 1))) :
    Ups t L z = Hs z + L * Ft t z + L * (2 * t - t ^ 2) := by
  have hF := Fmoments t z h1 hN.hasSum hT.hasSum
  unfold Ups Ft nbar Tz
  rw [hF.tsum_eq]
  ring

lemma summable_of_finite_support (f : ℕ → ℝ) (M : ℕ) (hf : ∀ n, M < n → f n = 0) :
    Summable f :=
  summable_of_ne_finset_zero (s := range (M + 1)) (fun n hn => hf n (by simp at hn; omega))

lemma zs_summable {s : ℝ} (hs0 : 0 ≤ s) (hs1 : s < 1) :
    HasSum (fun n => zs s n ^ 2) 1 ∧ Summable (fun n : ℕ => (n : ℝ) * zs s n ^ 2) ∧
      Summable (fun n : ℕ => ((n : ℝ) + 1) * zs s n * zs s (n + 1)) := by
  have hx0 : 0 ≤ s ^ 2 := sq_nonneg s
  have hx1 : s ^ 2 < 1 := by nlinarith
  have ha : 0 < 1 - s ^ 2 := by linarith
  have hg := hasSum_geometric_of_lt_one hx0 hx1
  have hn := hasSum_coe_mul_geometric_of_norm_lt_one
    (show ‖s ^ 2‖ < 1 by rw [Real.norm_of_nonneg hx0]; exact hx1)
  refine ⟨?_, ?_, ?_⟩
  · have := hg.mul_left (1 - s ^ 2)
    rw [mul_inv_cancel₀ ha.ne'] at this
    simpa [zs_sq hs1 hs0] using this
  · have := (hn.mul_left (1 - s ^ 2)).summable
    refine this.congr (fun n => ?_)
    rw [zs_sq hs1 hs0]; ring
  · have := ((hn.add hg).mul_left ((1 - s ^ 2) * s)).summable
    refine this.congr (fun n => ?_)
    unfold zs
    have hsq : √(1 - s ^ 2) ^ 2 = 1 - s ^ 2 := Real.sq_sqrt ha.le
    linear_combination (-((n : ℝ) + 1) * s ^ n * s ^ (n + 1)) * hsq

/-- The Beigi–Rahimi-Keshari input in Y form implies the sequence form `BRK`. -/
theorem brk_of_upsilon {t L : ℝ} (hU : BRKUpsilon t L) : BRK t L := by
  intro z hz0 ⟨M, hM⟩ h1 c0 hc0
  have hNz := summable_of_finite_support (fun n : ℕ => (n : ℝ) * z n ^ 2) M
    (fun n hn => by simp [hM n hn])
  have hTz := summable_of_finite_support (fun n : ℕ => ((n : ℝ) + 1) * z n * z (n + 1)) M
    (fun n hn => by simp [hM n hn])
  have key := hU z hz0 ⟨M, hM⟩ h1 (c0 + L * (2 * t - t ^ 2)) (fun s hs0 hs1 => by
    obtain ⟨a, b, c⟩ := zs_summable hs0 hs1
    rw [Ups_eq t L (zs s) a b c]
    linarith [hc0 s hs0 hs1])
  rw [Ups_eq t L z h1 hNz hTz] at key
  linarith

/-- **Theorem 1** with the Beigi–Rahimi-Keshari input in its Y form. -/
theorem theorem1_upsilon {q t : ℝ} (hq : 0 < q) (hqt : q < t) (ht : t ≤ 1)
    (hU : BRKUpsilon t (Thm2Cert.lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ) (G : ℝ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) G)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Thm2Cert.EE q ≤ ∑' n, negMulLog (c n ^ 2)
      + Thm2Cert.lam q t * ((G + 1 - t ^ 2) - Thm2Cert.dt t q) :=
  theorem1 hq hqt ht (brk_of_upsilon hU) c hc hc0 hnorm hH X Y G hG rX cX rY cY
    hrX hcX hrY hcY hX hY

/-- **Endpoint `q = t < 1`** (Eq. S:endpoint): a state with `D_t(ψ) = d_t(t) = 1 − t²`
(i.e. `G = 0`) has `E(ψ) ≥ 𝓛(t)`. -/
theorem theorem1_endpoint_state {t : ℝ} (ht0 : 0 < t) (ht1 : t < 1)
    (hU : ∀ q, 0 < q → q < t → BRKUpsilon t (Thm2Cert.lam q t))
    (c : ℕ → ℝ) (hc : Antitone c) (hc0 : ∀ n, 0 ≤ c n) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2)))
    (X Y : ℕ → ℕ → ℂ)
    (hG : HasSum (fun ij : ℕ × ℕ => ‖((t * c ij.1 : ℝ) : ℂ) * X ij.1 ij.2
      - (c ij.2 : ℂ) * Y ij.1 ij.2‖ ^ 2) 0)
    (rX cX rY cY : ℕ → ℝ)
    (hrX : ∀ i, 0 < c i → HasSum (fun j => ‖X i j‖ ^ 2) (rX i))
    (hcX : ∀ i, 0 < c i → HasSum (fun j => ‖X j i‖ ^ 2) (cX i))
    (hrY : ∀ i, 0 < c i → HasSum (fun j => ‖Y i j‖ ^ 2) (rY i))
    (hcY : ∀ i, 0 < c i → HasSum (fun j => ‖Y j i‖ ^ 2) (cY i))
    (hX : ∀ i, 0 < c i → rX i - cX i = 1) (hY : ∀ i, 0 < c i → rY i - cY i = 1) :
    Thm2Cert.EE t ≤ ∑' n, negMulLog (c n ^ 2) := by
  apply theorem1_endpoint ht0 ht1
  intro q hq hqt
  have h := theorem1_upsilon hq hqt ht1.le (hU q hq hqt) c hc hc0 hnorm hH X Y 0 hG
    rX cX rY cY hrX hcX hrY hcY hX hY
  have hdt : Thm2Cert.dt t t = 1 - t ^ 2 := by
    have : 1 - t ^ 2 ≠ 0 := by nlinarith
    unfold Thm2Cert.dt; field_simp
  rw [hdt]
```

```
  simpa using h

end Thm1Cert
```

Thm1Cert/Corollaries.lean

```
/-
Corollaries and the general-state limit for Theorem 1 of G. Adesso, "Optimality of Gaussian
entanglement of formation" (arXiv:2608.01909v3).

  * `theorem1_limit`: the affine bound passes to limits E_L → E, D_L → D. With the normalized
    Schmidt truncations C^{(L)} (finite rank, covered by `theorem1_upsilon` in the paired-basis
    matrix model), this gives Theorem 1 for every finite-energy state, the convergence
    G_t(C^{(L)}) → G_t(C), E(ψ_{C^{(L)}}) → E(ψ_C) being the Fock-space input of the SM
    (proof of Theorem S1, last paragraph);
  * `entropy_truncation`: E(ψ_{C^{(L)}}) → E(ψ_C) for the normalized truncations, at the level of
    Schmidt spectra (finite entropy);
  * `fixed_entanglement_endpoint` (q = t) and `separability_criterion` (q = 0, i.e. E(ψ) = 0
    implies D_t(ψ) ≥ 1), completing Eq. (S:fixedE) for 0 ≤ q ≤ t.
-/
import Thm1Cert.Upsilon

open Finset Real Filter Topology

set_option autoImplicit false

noncomputable section

namespace Thm1Cert

open Thm2Cert (EE gfun lam dt EE_formula lam_pos)

/-- **Limit step.** If the affine bound holds along a sequence of states with `E_L → E` and
`D_L → D`, it holds in the limit. -/
theorem theorem1_limit {q t : ℝ} (E D : ℝ) (EL DL : ℕ → ℝ)
    (h : ∀ L, EE q ≤ EL L + lam q t * (DL L - dt t q))
    (hE : Tendsto EL atTop (𝓝 E)) (hD : Tendsto DL atTop (𝓝 D)) :
    EE q ≤ E + lam q t * (D - dt t q) := by
  have hlim := hE.add ((hD.sub (tendsto_const_nhds (x := dt t q))).const_mul (lam q t))
  exact ge_of_tendsto hlim (Eventually.of_forall h)

/-- Entropy of the normalized Schmidt truncations converges: with `P_M = ∑_{n ≤ M} c_n²`,
`H((c^{(M)})²) = S_M/P_M + ln P_M → H(c²)`. -/
theorem entropy_truncation (c : ℕ → ℝ) (hnorm : HasSum (fun n => c n ^ 2) 1)
    (hH : Summable (fun n => negMulLog (c n ^ 2))) :
    Tendsto (fun M : ℕ => (∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) / (∑ n ∈ range (M + 1), c n ^ 2)
      + log (∑ n ∈ range (M + 1), c n ^ 2)) atTop (𝓝 (∑' n, negMulLog (c n ^ 2))) := by
  have hP : Tendsto (fun M : ℕ => ∑ n ∈ range (M + 1), c n ^ 2) atTop (𝓝 1) :=
    (hnorm.tendsto_sum_nat).comp (tendsto_add_atTop_nat 1)
  have hS : Tendsto (fun M : ℕ => ∑ n ∈ range (M + 1), negMulLog (c n ^ 2)) atTop
      (𝓝 (∑' n, negMulLog (c n ^ 2))) :=
    (hH.hasSum.tendsto_sum_nat).comp (tendsto_add_atTop_nat 1)
  have hlog : Tendsto (fun M : ℕ => log (∑ n ∈ range (M + 1), c n ^ 2)) atTop (𝓝 0) := by
    rw [← Real.log_one]
    exact (Real.continuousAt_log one_ne_zero).tendsto.comp hP
  have := (hS.div hP one_ne_zero).add hlog
  simpa using this

/-- **Eq. (S:fixedE) at `q = t`**: `D_t(ψ) = G + 1 − t² ≥ 1 − t² = d_t(t)` since `G ≥ 0`. -/
theorem fixed_entanglement_endpoint {t G : ℝ} (ht0 : 0 < t) (ht1 : t < 1) (hG : 0 ≤ G) :
    dt t t ≤ G + 1 - t ^ 2 := by
  have : 1 - t ^ 2 ≠ 0 := by nlinarith
  have hd : dt t t = 1 - t ^ 2 := by unfold dt; field_simp
  rw [hd]; linarith

lemma EE_nonneg {q : ℝ} (hq : 0 < q) (hq1 : q < 1) : 0 ≤ EE q := by
  unfold EE gfun
  set y := q ^ 2 / (1 - q ^ 2) with hy
  have hy0 : 0 < y := div_pos (by positivity) (by nlinarith)
  have h1 : log y ≤ log (y + 1) := Real.log_le_log hy0 (by linarith)
  have h2 : 0 ≤ log (y + 1) := Real.log_nonneg (by linarith)
  nlinarith

/-- **Separability criterion (`q = 0` in Eq. S:fixedE)**: if the affine bound holds for all
`0 < q < t` and `E(ψ) ≤ 0` (a product state), then `D_t(ψ) ≥ 1 = d_t(0)`. -/
theorem separability_criterion {t E D : ℝ} (ht0 : 0 < t) (ht : t ≤ 1)
    (h : ∀ q, 0 < q → q < t → EE q ≤ E + lam q t * (D - dt t q)) (hE : E ≤ 0) : 1 ≤ D := by
  have hle : ∀ q, 0 < q → q < t → dt t q ≤ D := by
    intro q hq hqt
    have hL := lam_pos hq hqt ht
    have h0 := EE_nonneg hq (lt_of_lt_of_le hqt ht)
    have := h q hq hqt
    by_contra hc
    rw [not_le] at hc
    have : lam q t * (D - dt t q) < 0 := mul_neg_of_pos_of_neg hL (by linarith)
    linarith
  have hcont : ContinuousAt (fun q => dt t q) 0 := by
    unfold dt
    exact ((continuousAt_const.sub (continuousAt_const.mul continuousAt_id)).pow 2).div
```

```
      (continuousAt_const.sub (continuousAt_id.pow 2)) (by norm_num)
  have hc := hcont.tendsto.mono_left (nhdsWithin_le_nhds (s := Set.Ioo 0 t))
  have hev : ∀ᶠ q in 𝓝[Set.Ioo 0 t] (0 : ℝ), dt t q ≤ D :=
    eventually_nhdsWithin_of_forall (fun q hq => hle q hq.1 hq.2)
  have : (𝓝[Set.Ioo 0 t] (0 : ℝ)).NeBot := by
    rw [← mem_closure_iff_nhdsWithin_neBot, closure_Ioo ht0.ne]
    exact ⟨le_rfl, ht0.le⟩
  have h1 := le_of_tendsto hc hev
  have hd0 : dt t 0 = 1 := by unfold dt; simp
  rwa [hd0] at h1

end Thm1Cert
```